

UCLA

UCLA Electronic Theses and Dissertations

Title

Adaptive Memory Management for Memory-Disaggregated Data Centers

Permalink

<https://escholarship.org/uc/item/6xc039n0>

Author

Liu, Shi

Publication Date

2025

Peer reviewed|Thesis/dissertation

UNIVERSITY OF CALIFORNIA
Los Angeles

Adaptive Memory Management for Memory-Disaggregated Data Centers

A dissertation submitted in partial satisfaction
of the requirements for the degree
Doctor of Philosophy in Computer Science

by

Shi Liu

2025

© Copyright by
Shi Liu
2025

ABSTRACT OF THE DISSERTATION

Adaptive Memory Management for Memory-Disaggregated Data Centers

by

Shi Liu

Doctor of Philosophy in Computer Science

University of California, Los Angeles, 2025

Professor Harry Guoqing Xu, Chair

Memory disaggregation has emerged as a promising strategy to address the pressing challenge of low memory utilization in modern data centers. While recent advancements in network technologies have enabled the decoupling of memory from compute resources, most existing solutions are designed around static assumptions on memory access behaviors, which neglect the diverse and dynamic behaviors exhibited by real-world applications. As a result, these systems struggle to deliver robust performance across varying workloads.

This dissertation presents a novel, adaptive approach to memory management for memory-disaggregated data centers. Rather than relying on fixed assumptions, the proposed techniques profile and react to changing memory access behaviors at runtime, making intelligent decisions on data placement, transfer granularity, and sharing mechanisms. This philosophy is embodied in two systems: Atlas, a hybrid data path that dynamically selects the most efficient remote data access granularity based on fine-grained, always-on profiling of memory locality; and Firework, a framework for distributed applications on CXL shared memory that adaptively manages data placement and sharing across host-local and device memory. Both systems leverage lightweight, online monitoring to respond to the dynamic and

input-dependent behaviors of data center applications, achieving significant improvements in throughput and efficiency compared to existing techniques.

Together, these contributions highlight the critical importance of adaptiveness in the design of disaggregated memory systems. By bridging the gap between hardware capabilities and dynamic application behaviors, this dissertation establishes a foundation for the next generation of efficient, scalable, and flexible memory management in data center environments.

The dissertation of Shi Liu is approved.

Ravi Netravali

Shan Lu

Jens Palsberg

Todd D. Millstein

Harry Guoqing Xu, Committee Chair

University of California, Los Angeles

2025

TABLE OF CONTENTS

1	Introduction	1
1.1	Challenges	2
1.2	Key Insight	5
1.3	Contributions	5
2	Background	8
2.1	RDMA-Based Remote Memory	8
2.1.1	Page-Based Data Transfer	9
2.1.2	Object Fetching	9
2.2	CXL Memory Devices	11
3	Atlas: Hybrid Data Path for Efficient Remote Memory	12
3.1	Overview	12
3.2	Motivation	17
3.3	Atlas Design and Implementation	21
3.3.1	Overview	21
3.3.2	Synchronization of the Two Paths	23
3.3.3	Memory Management	29
3.4	Evaluation	32
3.4.1	Setup and Methodology	32
3.4.2	Throughput	35
3.4.3	Latency	40

3.4.4	Performance Drill Down	42
3.5	Related Work	48
3.6	Summary	49
4	Firework: Efficient and Transparent Compute Scaling with CXL Shared Memory	50
4.1	Overview	50
4.2	Motivation	55
4.3	Firework Design and Implementation	57
4.3.1	Execution Model	57
4.3.2	Virtual Address Space	60
4.3.3	Page Sharing	61
4.3.4	Page Unsharing	63
4.3.5	Threading, Locking, and Memory Management	67
4.3.6	Remote I/O and Fault Tolerance	69
4.3.7	Dynamic Scaling	70
4.3.8	Limitations	70
4.4	Evaluation	71
4.4.1	Methodology	71
4.4.2	Throughput and Scalability	77
4.4.3	Comparing Firework with DSMs	79
4.4.4	Performance Drilldown	80
4.5	Related Work	82
4.6	Summary	84

5	Conclusion	85
5.1	Key Contributions	85
5.2	Future Directions	86

LIST OF FIGURES

3.1	Statistics of Metis PageViewCount (MPVC): (a) access patterns, (b) performance comparisons between AIFM and Fastswap, (c) comparisons of eviction throughput (dotted lines) and CPU usage (crosses and triangles) between AIFM and Fastswap, and (d) access patterns when input is changed to Wikipedia Italian [8]. For these experiments, 25% of the working set resides in the compute server’s local memory. Sequential accesses (due to skewness) in the Map phase are highlighted in red boxes in (a), while in (d) such patterns do not exist.	17
3.2	Atlas unique pointer metadata.	23
3.3	Dereferencing an Atlas unique pointer in a deref scope.	25
3.4	Throughput comparison between Atlas, Fastswap and AIFM with varying local memory ratios. “All Local” lines represent the performance of unmodified applications under 100% local memory.	36
3.5	(a) 90 th latency as a function of throughput; (b) Latency CDF under 0.23 MOPS offered throughput. FS stands for Fastswap.	40
3.6	(a) 90 th latency as a function of throughput; (b) Latency CDF under 1 MOPS offered throughput. FS stands for Fastswap.	41
3.7	The percentage of pages with PSF= paging in the memory footprint changes with the elapsed execution time.	42
3.8	Throughput comparisons of DataFrame (DF) and Web Service (WS) when Atlas and AIFM enable compute offloading. CO stands for variants with compute offloading.	43
3.9	Runtime overhead breakdown: overhead is calculated as the ratio between the extra execution time introduced and the execution time under 100% local memory.	44
3.10	Sensitivity of the CAR threshold.	47

3.11	Normalized throughput of Memcached workloads running on Atlas and Atlas-LRU under 25% local memory.	47
4.1	Memory sharing patterns vary across inputs and phases: (a) the number of shared pages changes with different skewness levels in the input key space of DataFrame —“High” refers to highly skewed input data; (b) the number of shared pages changes as execution phases change in Collaborative Filtering	56
4.2	The execution model of a Firework application. One process is launched on each node, powered by the Firework distributed runtime and a custom kernel. The page sharing map (PSM) is stored in device memory, acting as an extension of the per-process page table in each node kernel to store shared page metadata and synchronize page sharing operations.	58
4.3	Layout of the value fields of a PSM entry.	59
4.4	Page mapping examples under the global virtual address space. VP/PP denotes virtual/physical page.	61
4.5	State machine of a shared page.	66
4.6	(a)-(e) Throughput and scalability comparison among Firework, share-everything, and single-process, normalized to the throughput of a single process with 4 cores (<i>i.e.</i> , the dark green dash line). (f) scalability comparisons between Firework, DRust [82], and GAM [22] on Feed Generation	76
4.7	(a) single-threaded page sharing and unsharing throughput under different batch sizes; (b) CF performance comparison between Firework, Firework without unsharing, and single-process.	79

LIST OF TABLES

3.1	Applications used for Atlas evaluation.	33
3.2	Major runtime overheads, operations involved in each type, and their affected systems.	44
4.1	Performance of different memory topologies. R/W stands for read/write. . . .	73
4.2	Applications used for Firework evaluation.	73

ACKNOWLEDGMENTS

At the conclusion of this five-year journey, I would like to take a moment to express my deepest gratitude to the many invaluable individuals who have provided constant support, kindness, and encouragement, making my Ph.D. experience both fulfilling and enjoyable.

First and foremost, I extend my heartfelt appreciation to my advisor, Harry Xu. Harry has consistently provided me with unwavering support, guiding me expertly through numerous technical challenges, and generously imparting invaluable life lessons that have profoundly shaped my personal and professional growth. It has been an incredible honor and privilege to have Harry as my advisor, and I know that my Ph.D. journey would have been far less enriching without his mentorship.

I would also like to sincerely thank my doctoral committee members: Jens Palsberg, Todd D. Millstein, Shan Lu, and Ravi Netravali, for their invaluable guidance and thoughtful feedback throughout the development of this dissertation and during my research presentations. Special thanks go to Shan Lu for her insightful comments and collaboration on Atlas and MemLiner, two of my favorite projects of my doctoral research.

My accomplishments during this journey would not have been possible without my exceptional collaborators. I am especially grateful to have worked alongside the outstanding researchers at UCLA SOLAR lab: Chenxi Wang, Haoran Ma, Yifan Qiao, Shu Anzai, Shan Yu, Christian Navasca, Jonathan Eyolfson, John Thorpe, Arthi Padmanabhan, Pengzhan Zhao, Zhenting Zhu, Jiyuan Wang, Yicheng Liu, Khanh Nguyen, Yuanqi Li, and many others. Additionally, I am honored and thankful for the opportunity to collaborate with brilliant colleagues from around the world, including Lei Chen, Zhenyuan Ruan, Qingda Lu, Yuanjiang Ni, Miryung Kim, Yiyang Zhang, Michael D. Bond, Stephen M. Blackburn, and everyone who has generously shared insights and support throughout my research endeavors.

I also deeply appreciate my friends Junyang Shao, Longjie Guo, Ru Wang, Qiliang Lai, Zhiming Xu, Yueyu Hu, Oscar Lin, Deming Yin, Hongyu Zhang, Lei Ruan, and Jiawei Ji,

whose companionship has brought me tremendous joy, refreshed my spirit, and continually inspired me to move forward.

Finally, I dedicate this dissertation to my mother, who has always stood by my decisions with unwavering support and unconditional love. Her invaluable lessons and endless encouragement have empowered me to face every challenge with confidence and resilience. Thank you for always being there.

VITA

Ph.D. Candidate in Computer Science
University of California, Los Angeles

2020 - 2025

B.Eng. in Computer Science and Technology
University of Science and Technology of China

2016 - 2020

PUBLICATIONS

Lei Chen, **Shi Liu (co-first author)**, Chenxi Wang, Haoran Ma, Yifan Qiao, Zhe Wang, Chenggang Wu, Youyou Lu, Xiaobing Feng, Huimin Cui, Shan Lu, Harry Xu “A Tale of Two Paths: Toward a Hybrid Data Plane for Efficient Far-Memory Applications.”, OSDI, 2024

Haoran Ma, Yifan Qiao, **Shi Liu**, Shan Yu, Yuanjiang Ni, Qingda Lu, Jiesheng Wu, Yiyang Zhang, Miryung Kim, Harry Xu, “DRust: Language-Guided Distributed Shared Memory with Fine Granularity, Full Transparency, and Ultra Efficiency”, OSDI, 2024

Chenxi Wang, Yifan Qiao (co-first author), Haoran Ma, **Shi Liu**, Yiyang Zhang, Wenguang Chen, Ravi Netravali, Miryung Kim, Guoqing Harry Xu, “Canvas: Isolated and Adaptive Swapping for Multi-Applications on Remote Memory”, NSDI, 2023

Haoran Ma, **Shi Liu**, Chenxi Wang, Yifan Qiao, Michael D Bond, Stephen M Blackburn, Miryung Kim, Guoqing Harry Xu, “Mako: a low-pause, high-throughput evacuating collector for memory-disaggregated datacenters”, PLDI, 2022

Chenxi Wang, Haoran Ma (co-first author), **Shi Liu**, Yifan Qiao, Jonathan Eyolfson, Christian Navasca, Shan Lu, Guoqing Harry Xu, “MemLiner: Lining up Tracing and Application for a Far-Memory-Friendly Runtime”, OSDI, 2022, **Best Paper Award**

Chenxi Wang, Haoran Ma, **Shi Liu**, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D Bond, Ravi Netravali, Miryung Kim, Guoqing Harry Xu, “Semeru: A Memory-Disaggregated Managed Runtime”, OSDI, 2020

CHAPTER 1

Introduction

Data centers have dramatically reshaped modern computing over the past few decades, powering services essential to everyday life—from web search, e-commerce, and social media to AI workloads. Yet, as one of the critical hardware resources within data centers, memory remains both overtaxed and underutilized. On one hand, memory demand has soared alongside the rise of data-intensive workloads. Applications like big data analytics and large language model inference now dominate memory footprints, turning memory into a primary cost driver. In some enterprises, these demands have pushed memory to account for nearly half of total data-center expenses, as seen at Microsofts Azure (approximately 50%) [1] and Meta (around 37%) [85]. On the other hand, memory utilization within these facilities remains surprisingly low. Analysis of resource trace data from Google and Alibaba indicates an average memory use of just around 40-60% across a cluster [109]. With memory representing a huge portion of data-center capital and ongoing operational costs, even slight utilization improvements translate into massive returns. One study estimates that a mere 1% uplift in memory efficiency could save over 7 billion dollars and reduce energy consumption by 140 million kilowatt-hours annually [28, 45, 57, 116]. These pressing issues cause high financial and energy waste and urge a solution for better efficiency.

The major reason behind the underutilization is the monolithic server architecture prevalent in today’s data centers, where each compute node hosts both CPU and memory in a tightly coupled bundle. As a job requires CPU and memory resources from the same machine, when one of the resources is saturated on each machine, the other resource becomes wasted as it cannot be allocated along with idle resources located on other servers. Mono-

lithic servers also introduce other significant limitations: Poor elasticity makes the hardware configuration hard to change after deployment. The tight bounding of different resources means that the failure of one resource makes the whole server unusable, causing a coarse fault domain. [109] All of these limitations urge an evolution of the memory architecture. One promising solution is memory disaggregation.

Memory disaggregation fundamentally decouples memory from compute, reorganizing data-center architectures into two loosely coupled but interconnected domains: compute nodes and memory nodes, linked via a high-performance network. In this topology, compute servers can directly access memory pools located anywhere on the network instead of being limited to their local DRAM. As a result, memory capacity becomes independently scalable, instead of being stuck with the amount determined when the compute resource was first set up. Disaggregated memory can be realized through distinct mechanisms depending on the interconnect technology. For example, remote memory leverages existing server hardware and RDMA-capable networks (e.g., InfiniBand, RoCE), allowing CPUs to perform one-sided reads and writes to remote memory on other servers with minimal OS overhead. On the contrary, Compute Express Link (CXL) [3] offers a clean-slate disaggregation paradigm. CXL enables cache-coherent, load/store access to dedicated memory devices—memory blades—via switches and controllers running on an extended version of PCIe protocols. Multiple hosts can concurrently access memory with strict coherence guarantees and zero redundant copies.

1.1 Challenges

As a novel memory architecture, memory disaggregation introduces unique challenges for memory management, due to two major observations:

Performance-Sensitive Memory Decisions Disaggregated memory introduces new hierarchical layers far beyond the traditional host DRAM with dramatic performance dis-

parities: accesses to RDMA-based remote memory can be 10-40× slower than local DRAM; CXL memory devices also incurs a 2-5× higher latency, while only providing 8%-20% of peak throughput compared with local DRAM. This huge performance difference makes memory decisions such as data placement and data transfer between different tiers much more critical to system performance. If data is fetched in an inefficient way, or data is stored in a memory backend slower than desired, the performance of the whole system would be directly impacted by the extended memory access latency. Therefore, to preserve performance for workloads running on disaggregated memory, the memory manager needs to be aware of the performance characteristics of different memory layers, and make wise data placement and transfer decisions to minimize overheads introduced by extended latencies and extra network traffic [15, 24, 46, 49, 80, 101, 103, 120–122].

Diverse and Dynamic Memory Access Behaviors Although a wide range of systems have been proposed to improve memory efficiency under a disaggregated setting, these solutions still fall short of providing consistent performance benefits on different data center workloads. The reason is that such systems often make assumptions about the memory access behaviors of target applications and optimize the system based on such assumptions. However, real-world data center workloads exhibit vastly diverse and highly dynamic memory access behaviors, which make them difficult if not impossible to predict via static configurations. To illustrate this point, we use memory access patterns as an example of memory access behaviors, to explain why it is diverse and dynamic, as well as how such features affect the performance of disaggregated memory systems. Note that memory access patterns are only one type of memory access behavior, which also includes hotness, mutability, access sizes, sharing patterns, consistency and synchronization needs, and much more.

First, memory access patterns are diverse in nature, due to the varying characteristics across different data center applications. For example, DataFrame [87] vector arrays can be streamed through during a scan; while the hash table bucket array in an in-memory

store such as Memcached [7] usually exhibits random or skewed access patterns. A research paper profiling 38 data center applications (spanning across data analytics, AI, HPC, graph processing, and more) reveals significant variations in per-structure access patterns [61].

Second, memory access evolves over time within a single execution. For instance, many MapReduce applications exhibit a phase-shifting access pattern: the Map phase is characterized by random hash table insertions, while the Reduce phase transitions to streaming-style accesses when aggregating buckets. Likewise, input datasets further introduce variability. For example, whether the input data is skewed for a hash table directly affects whether the buckets will contain memory hot spots. Thus, identical code can produce dramatically different memory behaviors.

Given the diverse and dynamic memory access patterns, when the accessed data is in remote memory, the memory access patterns are highly related to memory locality, and thus affect how the remote data should be fetched to local memory: for data with good locality, remote data should be fetched in bulks to improve fetching throughput, as most fetched data will also be accessed in the near future; while for data with bad locality, only the accessed data should be fetched, so network bandwidth and local memory are not wasted for data that will not be accessed. However, existing systems usually assume a fixed data transfer granularity, which leaves them with suboptimal data transfer efficiency. Our experiments show that choosing the appropriate data transfer strategy can boost performance: when running a MapReduce application on data paths fetching data with different granularities, the random Map phase sees 1.6 $\times$ throughput uplift with fine-grained fetching, while the streaming Reduce phase performs 3.3 $\times$ faster when using bulk fetching. Detailed analysis can be found in §3.2.

In summary, memory access behaviors define how memory should be managed in a memory-disaggregated setting. However, due to the dynamic nature of the memory access behaviors, the optimal decisions cannot be made statically.

1.2 Key Insight

This dissertation proposes an adaptive approach when building data paths for memory disaggregation. Instead of making static assumptions on the memory access behaviors, the design of the data path should be aware of the behaviors that are critical to the performance, and provide mechanisms that can act adaptively to fit different memory access behaviors with optimal efficiency.

Such adaptiveness requires the system to be able to identify the dynamic behaviors. Although there have been attempts to profile memory access behavior information with offline dry runs [49], such approaches introduce high profiling overhead, and behave poorly when dealing with memory access behavior diversity introduced by varying input, as the profiling result only applies to the input used by the dry run. Therefore, behavior profiling should be done online while the application is running to ensure the results always align with the current status of the program. Ideally, the profiling also needs to be lightweight, so the performance benefits introduced by adaptiveness can be preserved.

1.3 Contributions

This dissertation showcases two systems that are built based on the philosophy of adaptiveness. They each build upon one type of disaggregated memory architecture and explore adaptiveness based on one type of memory access behavior that is critical to system performance.

Chapter 3: Atlas: Hybrid Data Path for RDMA Remote Memory. Atlas explores how to improve data transfer efficiency for RDMA remote memory with an adaptive data path based on dynamic memory access patterns. Data residing in RDMA remote memory cannot be directly accessed with CPU load/store instructions. Therefore, a mechanism is needed to transfer data between local DRAM and remote memory. Existing solutions have

been categorized into two types, where the main difference between them is the granularity, or the unit of data transfer between local and remote memory. One type of data path transfers data in coarse-grained 4 kilo-byte pages, while another type transfers data in fine-grained objects. As discussed in §1.1, each transfer granularity only fits data on one side of the locality, introducing a large overhead when data is fetched with the undesired granularity. Ideally, each remote data access should be performed by the desirable data path, based on its locality. Atlas is built to achieve this goal with three key components: 1) a hybrid data path that is capable of fetching remote data with variable granularities; 2) a locality profiler that decides the desirable fetching path for every remote access; 3) a unified page eviction path to evict cold data from local DRAM to remote memory efficiently.

Chapter 4: Firework: Intelligent Data Sharing with CXL Shared Memory. Firework addresses the challenges to efficiently run distributed applications on CXL shared memory, with an adaptive data sharing mechanism based on dynamic memory sharing patterns. CXL shared memory unlocks new opportunities to design efficient cross-node shared memory systems, where compute on different hosts can concurrently access shared memory regions with hardware-enforced cache coherency, while still having access to local DRAM that is private to each host with much better performance. The challenge here is how to ensure that data shared across hosts is globally accessible, while the overhead introduced by the shared memory is minimized. A strawman solution to allocate all data in the shared memory incurs substantial performance overheads (e.g., more than 10×), while offloading the task to mark memory sharing patterns to the programmer is impractical for large-scale applications, and inherently suboptimal due to the dynamicity of memory sharing patterns. Firework aims to provide an efficient and transparent answer to the data-sharing challenge for CXL shared memory. It has two core capabilities: 1) a mechanism to migrate data between host-local DRAM and the shared memory device at the page granularity with full transparency; 2) an adaptive data sharing mechanism to make decisions about when and what data should be shared or stop being shared to optimize performance.

Dissertation Organization

The rest of the dissertation is organized as follows: Chapter 2 provides basic knowledge related to RDMA remote memory and CXL shared memory. Chapter 3 and 4 present the details of Atlas and Firework respectively. Finally, the conclusion and future directions are discussed in Chapter 5.

Bibliographic Notes

Parts of the dissertation are based on projects that I have conducted and published guided by my advisor Harry Xu, and support from other collaborators. Specifically, Chapter 3 extends a conference paper published in OSDI '24 (DOI: 10.5555/3691938.3691943), in which I was a co-first author and collaborated with Lei Chen, Chenxi Wang, Haoran Ma, Yifan Qiao, Zhe Wang, Chenggang Wu, Youyou Lu, Xiaobing Feng, Huimin Cui, Shan Lu, and Harry Xu [30].

CHAPTER 2

Background

2.1 RDMA-Based Remote Memory

Network advancements such as RDMA [41] make it possible for CPUs on one physical server to access the memory data on another physical server, a technique we refer to as remote memory (or far memory). The setting of remote memory uses the abstraction of CPU servers, which are abundant in compute resources and limited memory, and memory servers, which have little or no compute but are equipped with a large amount of memory.

A practical setting usually maintains the local DRAM in CPU servers as a cache of the application working set, while all data that cannot fit in the local DRAM resides on the memory server. Remote data are fetched to the local memory upon access, and local data is evicted to remote memory when considered cold under memory pressure. The reason behind such a design is twofold: 1) remote memory can only be accessed by RDMA primitives, not CPU load/store instructions. To maintain a uniform access interface for both local and remote data, remote data is first fetched to the local cache, so it can also be accessed with CPU instructions; 2) accesses to remote memory incur a much higher latency than local DRAM (e.g., 10-40×). By keeping hot data in local memory, the system can ensure memory performance for the majority of memory accesses.

How data is transferred between the local cache and remote memory is the key mechanism for a remote memory data path. There have been two types of data paths: one type utilizes the existing paging system in the OS kernel to transfer data at the page granularity, while

the other type implements new runtime systems to enable the transfer of individual objects.

2.1.1 Page-Based Data Transfer

The page-level data path [15, 46, 84, 101, 122] piggybacks on the paging system in the OS kernel. The paging system is originally designed to store cold pages to a swap partition such as a disk under memory pressure. By mapping remote memory as a swap partition of the compute server, the swap system can evict cold pages based on a page-level LRU algorithm. When a remote page is accessed, a page fault is triggered to bring the page back to the local memory. The whole process is maintained by the OS virtual memory system and transparent to the application. It is also an efficient mechanism to transfer bulk data between local and remote memory when most content on a page should be transferred to the other memory backend. However, since it must transfer all data on the same page together, it may fetch useless objects or evict hot objects, leading to a waste of network bandwidth and local memory, a problem called I/O amplification.

2.1.2 Object Fetching

Object fetching is motivated by two observations on the inefficiencies of paging. First, fetching data at the page granularity often leads to I/O amplification [24]. Second, managing data in the kernel space is agnostic to program semantics, resulting in missed optimization opportunities [103, 119, 121]. As such, work has been proposed to manage data with a language runtime at a finer-grained object (or cache-line) granularity [24, 81, 103, 120, 121, 123]. Unlike paging, the runtime can only manage objects in user space, which results in two consequences: (1) the runtime must change the virtual address of an object when moving it and hence must change all its pointers; and (2) the runtime must maintain all metadata itself (*e.g.*, LRU), which used to be maintained by the kernel. Here we discuss AIFM [103] as an example of object fetching. AIFM proposes two abstractions for developers to manage remote memory: *remoteable pointer* and *dereference scope*.

Remoteable pointer. AIFM extends the *smart pointer* abstraction of C++ to implement remoteable pointers (`RemPtr`) for remote data management. There are two types of `RemPtr`: 64-bit unique remoteable pointers (similar to `std::unique_ptr`) and 128-bit shared remoteable pointers (similar to `std::shared_ptr`). Developers need to explicitly declare data as remote type and manage them via the `RemPtr`. For example, each unique `RemPtr` has 64 bits—the lower 47 bits are used as the virtual address of the data, and the upper 17 bits are used to record metadata, such as dirty (D), present (P), hot (H), evacuated (E), *etc.* When accessing data via a `RemPtr`, AIFM checks the metadata of the `RemPtr` to detect its status, *e.g.*, checking the P bit to see if the object is in local memory. Next, AIFM masks the `RemPtr` to obtain the actual virtual address.

Dereference scope. Each smart pointer dereference and subsequent raw pointer accesses must be enclosed by a dereference scope, which works as an *evacuation fence* to guarantee correctness. AIFM performs periodical *concurrent object evacuation* that swaps out cold objects to remote memory and compacts local memory to improve data locality. It is challenging to move objects when they are being used by other threads since moving objects requires updating all their pointers. Smart pointers solve this problem because these pointers can be recorded in object headers and updated after moves are conducted. However, an application may read raw pointers from smart pointers and store them in registers or on the stack, which cannot be updated by the runtime.

To guarantee correctness for pointer updating, AIFM requires developers to explicitly declare dereference scopes for each object, which define where raw pointers of the object may exist. Evacuation of the object never happens concurrently with the execution of any of its dereference scopes that started before the evacuation decision. A dereference scope serves as a synchronization mechanism between an event that moves the object and another that uses it.

2.2 CXL Memory Devices

Compute Express Link (CXL) [3] is an open interconnect standard built on the PCIe protocol, designed to enable high-speed, efficient communication between CPUs and devices. CXL defines three main device types. Within this dissertation, we focus on Type 3 devices, which serve as memory expanders that allow the host to access and manage additional memory provided by the device. We refer to physical servers as nodes, and their internal memory as local or private memory, in contrast to device memory or shared memory provided by CXL devices. Device memory can be mapped into a node’s virtual address space and accessed using regular load/store instructions, just like local memory. CXL 3.0 further enhances this model by introducing *back-invalidation*, a hardware mechanism that ensures cache coherence across the CPUs accessing the device. CXL 3.0 provides hardware-supported cache-coherent shared memory, allowing multiple nodes to access and operate on shared data directly—eliminating the need for copying and passing data between them. With CXL, the key distinctions between private and device memory lie in access performance. Accessing device memory involves additional fabric layers compared to private memory, resulting in higher latency and lower throughput. At a high level, CXL memory can introduce 1.5-10× higher latency and reduce throughput by 2-13×, depending on how far it is from the accessing process (*i.e.*, topology). CXL memory performance details can be found in §4.4.1.

CHAPTER 3

Atlas: Hybrid Data Path for Efficient Remote Memory

Existing remote memory systems fall into one of two data paths: one that uses the kernel’s paging system to transparently access remote memory at the page granularity, and a second that bypasses the kernel, fetching data at the object granularity. While it is generally believed that object fetching outperforms paging due to its fine-grained access, it requires significantly more compute resources to run object-level LRU and eviction.

This chapter introduces Atlas, a hybrid data plane enabled by a runtime-kernel co-design that simultaneously enables accesses via these two data paths to provide high efficiency for real-world applications. Atlas uses *always-on* profiling to continuously measure page locality. For workloads already with good locality, paging is used to fetch data, whereas for those without, object fetching is employed. Object fetching moves objects that are accessed close in time to contiguous local space, dynamically improving locality and making the execution increasingly amenable to paging, which is much more resource-efficient. Our evaluation shows that Atlas improves the throughput (*e.g.*, by 1.5× and 3.2×) and reduces the tail latency (*e.g.*, by one and two orders of magnitude) when using remote memory, compared with AIFM and Fastswap, the state-of-the-art techniques respectively in the two categories.

3.1 Overview

Thanks to the high bandwidth and low latency provided by modern network fabrics such as InfiniBand, remote memory techniques [15, 46, 103, 121–123] enable an abstraction of unlimited memory for applications by allowing them to use available memory on remote

servers, thereby simultaneously improving application performance and datacenters’ overall memory utilization.

Although techniques such as RDMA enable fast network access, each remote access is still at least an order of magnitude slower than a local access. As such, it is paramount to optimize the remote access data plane so that applications can benefit from increased memory capacity without suffering a significant performance hit. A major line of work for accessing remote memory is using the kernel’s paging system, exemplified by techniques such as InfiniSwap [46], Fastswap [15], Canvas [122] and Hermit [101]. These techniques allow applications to transparently access remote memory at the *page* granularity, using the kernel’s swap system to swap pages in and out between local and remote memory.

While paging works well for applications that perform bulk data movement and exhibit clear (sequential or strided) access patterns, its coarse granularity incurs substantial *I/O amplification* (*i.e.*, pages loaded only contain a small amount of useful data) for applications that exhibit irregular (or random) access patterns, such as Memcached [7] and graph applications [65]. To reduce I/O amplification, a recent line of work exemplified by AIFM [103] and Kona [24] advocates to access data at a much finer (object) granularity using a user-space runtime system. Swapping objects, rather than pages, can significantly reduce the amount of useless data swapped, leading to higher efficiency. Furthermore, since objects are the data abstraction for developers to write programs, they carry semantics (*i.e.*, user intention) that can be exposed to and used by the runtime to perform additional optimizations, such as data-structure-based prefetching.

Fetching objects at runtime, however, comes at a cost. A drawback that was often overlooked by existing works is that object fetching requires *non-trivial compute resources* to profile object usage, identify patterns, and perform object-level LRU and eviction. For instance, running an object-level LRU algorithm is **one order of magnitude** more expensive than page-based LRU due to a huge number of objects to be processed and the lack of hardware support for tracking object accesses. This overhead is significantly more pronounced

in real-world scenarios where CPUs are all busy with executing application threads—given a tight time budget, memory management threads cannot scan enough objects to make accurate LRU and eventually have to evict arbitrary objects.

As a result, the right access mechanism is essentially the result of a tradeoff between program locality (*i.e.*, how bad I/O amplification can be) and the amount of compute resources available (*i.e.*, how many cores can be dedicated to object-level memory management tasks). For programs with poor locality, the overhead of object-level memory management can be offset from the large gains of reducing I/O amplification. On the other hand, for programs with good locality and insignificant I/O amplification, the overhead of object fetching stands out, especially in an environment where applications have taken all compute resources (see §3.2).

There is a recent line of compiler-based techniques (as exemplified by Mira [49]) that profile a program *offline* to understand such a tradeoff, so that compiler can statically choose the mechanism for each data access when compiling the program. However, offline profiling hinges upon program input. For interactive applications such as Memcached, their input data comes from users and keeps changing, rendering a dry-run-based technique ineffective.

Major Insight. The main question we ask here is: can we enable *always-on* profiling for an application to identify its access patterns and dynamically switch between paging and object fetching to adapt to the observed patterns? This approach, if implemented efficiently, has two advantages over the state-of-the-art techniques. First, its continuous profiling identifies patterns *on-the-fly* for different computation stages or parallel threads accessing different data structures, even if the program input keeps changing. As a result, it can quickly change the access path to use a more efficient fetching mechanism. Second, for programs with irregular patterns, object fetching moves objects that are accessed close in time into contiguous memory space, dynamically improving locality as the program executes. This makes it possible for the execution to *be increasingly amenable to paging*, which has higher

resource efficiency (see §3.2).

Although promising, realizing this insight requires overcoming three major challenges, as elaborated below:

The *first challenge* is how to continuously and accurately profile an application with low overhead. Kernel-based page-level profiling, though efficient, does not provide sufficient information with respect to fine-grained data locality. For example, if one single hot object on a page keeps getting accessed but none of other objects do, the kernel-based profiling would identify the page as a hot page although the page clearly possesses poor locality and its accesses should go through object fetching, not paging.

To enable fine-grained profiling, Atlas divides a page into a set of *cards*, each of which is a unit for our locality measurement. We leverage the runtime (and in particular, a *read barrier*) to compute a *card access table (CAT)* (§3.3.3) for each page, which is a bitmap where each bit corresponds to a card (*i.e.*, consecutive 16 bytes) on the page and a set bit represents that the card has been accessed since the page was allocated or last swapped in. A page with a high *card access rate* (CAR, measured as the percentage of the set bits in its CAT) is deemed to possess good locality and should be accessed with paging, while a page with a low CAR has poor locality and should be accessed with object fetching.

The *second challenge* is how to dynamically switch access mechanisms. Atlas uses a read barrier at each smart pointer dereference. The barrier quickly checks a per-page *path selector flag* (PSF) for the remote page to be accessed. Each PSF is a 1-bit flag, set to either **runtime** or **paging**. **runtime** indicates that the runtime path should be used to fetch individual objects (like AIFM), while **paging** means that the paging path is taken to fetch an entire page. The PSF of a page is updated only when the page is evicted based upon the page’s CAR—it is set to **runtime** if the page’s CAR is low, indicating the page exhibits poor locality, and **paging** otherwise, indicating good locality.

Although Atlas supports both object fetching and paging at *ingress*, it evicts data only at the page granularity at *egress*, to reduce the high overhead associated with object-level

profiling and LRU. While evicting pages may introduce I/O amplification for workloads with poor locality, this impact is insignificant under Atlas, because accesses in these workloads would likely go through the object fetching path, which improves locality by moving objects accessed close in time into contiguous local space. The enhanced locality effectively mitigates the negative impact of page-level eviction.

To reduce fragmentation resulting from dead objects, Atlas runs *concurrent evacuation* tasks that periodically move live objects into contiguous memory space. During each evacuation, Atlas groups recently-accessed objects into contiguous pages to further improve data locality.

The *third challenge* is how to synchronize the two access paths. Since the kernel and the runtime are not coordinated (*e.g.*, the kernel does not inform the runtime of the start or the completion of a page-fault handling), special care must be taken to prevent the two access paths from creating inconsistent data copies. In particular, correctness issues may arise from a set of ingress and egress events (*i.e.*, object-in, page-in, and page-out) that occur simultaneously. Atlas solves the problem with a synchronization protocol (see §3.3.2), implemented with a combination of runtime and kernel support.

Results. We have evaluated Atlas with a set of eight applications that cover a full range of memory access patterns: sequential, random, and mixed. Our results show that Atlas enables these applications running on remote memory to achieve an overall of 1.5× and 3.2× throughput improvement, compared with AIFM [103] and Fastswap [15], respectively. Atlas reduces the tail latency by one and two orders of magnitude when compared with AIFM and Fastswap. Atlas is available at <https://github.com/wangchenxi7/Atlas>.

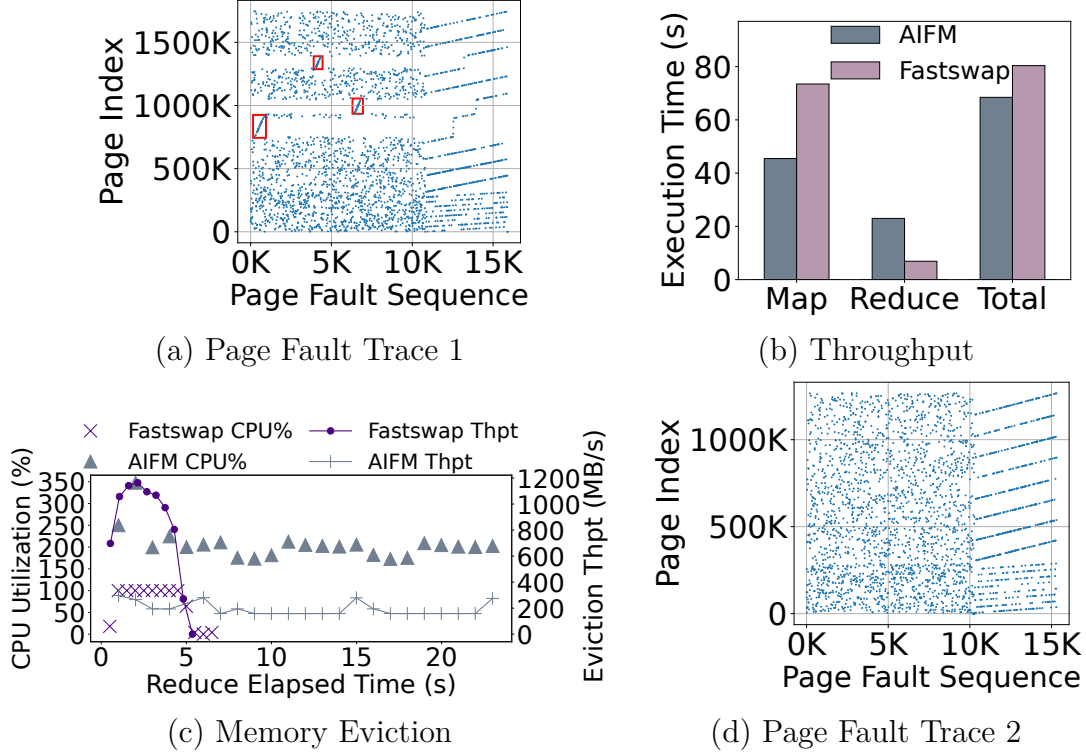


Figure 3.1: Statistics of Metis PageViewCount (MPVC): (a) access patterns, (b) performance comparisons between AIFM and Fastswap, (c) comparisons of eviction throughput (dotted lines) and CPU usage (crosses and triangles) between AIFM and Fastswap, and (d) access patterns when input is changed to Wikipedia Italian [8]. For these experiments, 25% of the working set resides in the compute server’s local memory. Sequential accesses (due to skewness) in the Map phase are highlighted in red boxes in (a), while in (d) such patterns do not exist.

3.2 Motivation

This section motivates the necessity of a hybrid data plane. We first demonstrate the diverse memory access patterns of real-world cloud applications and explain the underlying reasons. Next, we compare fetching performance between using a runtime and the kernel’s paging system. For the runtime approach, we re-implemented applications with AIFM [103]. For paging, we used Fastswap [15]. Finally, we discuss the opportunities provided by *dynamic* path switching.

Diverse memory accesses. Real-world applications exhibit complicated memory access patterns, which are a combination of multiple primitive patterns such as sequential, strided, skewed, and random. Access patterns depend on at least two factors: (1) the computation model and (2) the data model. Next we elaborate on these factors:

On one hand, many applications are phase-changing and each phase follows a distinct computation model. On the other hand, the same phase may exhibit varied access patterns when processing different data structures.

An example is data-processing applications [133, 135] that implement MapReduce. We experiment with Metis [83], a MapReduce framework optimized for multicore architectures, with a Page View Count (PVC) program [55, 102] and report its page fault sequence in Figure 3.1(a). Since PVC is executed with 8 cores, we launch 8 threads for each (Map or Reduce) phase to exploit data parallelism. During the Map phase, each thread loads chunks of input data from the disk and initializes loaded website URLs and users as memory data. Next, PVC shuffles URLs into different buckets of a hash table based on their hash values. The Reduce phase scans each entry to count each URL’s users.

The left/right part of Figure 3.1(a) illustrates the page fault sequence of the Map/Reduce phase. The Map phase (left) inserts URLs into the hash table, and accesses there are mostly random. However, given that the dataset used to run this program is *skewed*, there are several ranges of sequential accesses in the Map phase, as highlighted in the boxes (*i.e.*, certain hash buckets are much larger than others and hence traversing these buckets exhibits sequential patterns). During the Reduce phase (right), each task that aggregates users of URLs scans entries in a bucket sequentially, resulting in a clear sequential access pattern, as shown in the second (right) half of Figure 3.1(a).

Granularity-performance tradeoff. Object fetching minimizes I/O amplification by fetching fine-grained objects [23, 103]. However, compared to paging, object fetching does not always show clear benefits—for workloads with good locality, data on the same pages

are accessed close in time and the kernel can already effectively and accurately prefetch data. When benefits are insignificant, the overhead for object-level memory management stands out. To compare fetching efficiency between the runtime and the kernel, we run the Metis PVC benchmark on AIFM and Fastswap, respectively. Figure 3.1(b) reports their performance comparisons.

Since a MapReduce program has clear phases, we broke down the execution time into Map and Reduce. AIFM outperforms Fastswap by 1.6 $\times$ in the Map phase due to object fetching—most remote accesses in Map are random as words are inserted into different buckets of the hash map. On the contrary, AIFM underperforms Fastswap by 3.3 $\times$ in the Reduce phase, which exhibits clear sequential patterns.

Object eviction cost. The main reason why object fetching underperforms paging for programs with good locality is the high cost associated with profiling objects and maintaining object-based LRU for eviction. For example, eviction must be done quickly as it blocks further memory allocations [101]. As a result, AIFM constantly maintains dozens of profiling/eviction threads to track the hotness of (billions of) objects and evict cold objects. However, if these threads cannot obtain enough CPU resources from the application, they end up scanning only a small percentage of objects before time runs out and then evict objects with limited hotness information, resulting in data thrashing (*i.e.*, hot objects get swapped out and quickly swapped back in).

Figure 3.1(c) compares the eviction throughput and CPU utilization for eviction of AIFM and Fastswap during the Reduce phase. AIFM continuously performs object-level hotness tracking and eviction with around 200% (up to 350%) CPU usage in the entire Reduce phase. On the contrary, Fastswap finishes most of the page eviction task within the first five seconds and consumes no more than 100% CPU resources during the eviction. Overall, Fastswap consumes *an order of magnitude* less compute (cycles) than AIFM for eviction over the Reduce phase. Even with significantly fewer CPU resources, Fastswap’s eviction

throughput is still $\sim 5\times$ higher than that of AIFM, due to the low memory management cost associated with paging.

Necessity of online profiling and path switching. Offline profiling techniques [49, 69, 78, 100, 126] were proposed to analyze program semantics and data accesses. However, these techniques are ineffective in identifying the optimal solution for a real-world application for two major reasons.

On the one hand, even if the application’s computation phases may be analyzed by an offline profiling technique, its access patterns can change dramatically in response to *inputs*. As Figure 3.1(d) demonstrates, when fed with a different dataset (which does not exhibit skewness), the program’s access patterns change significantly—*e.g.*, due to the lack of skewness, the Map phase no longer exhibits sequential patterns. In fact, for any interactive applications including Memcached [7], DataFrame [87], or streaming data systems [35, 65, 118], their behaviors and access patterns vary significantly with different user requests and workloads.

On the other hand, as discussed earlier, object fetching consumes extensive CPU resources. This may be acceptable when CPU resources are not fully saturated but becomes problematic as soon as all CPU cores are occupied (*e.g.*, another tenant starts using the server). Clearly, offline profiling is not able to predict such environmental changes.

These issues necessitate a dynamic technique that can continuously profile program executions and perform runtime data path switching as new behaviors and/or environmental changes are detected. Our main objective is to use object fetching to minimize I/O amplification and enhance locality, paving the way for subsequent accesses to operate on data with established locality and thus benefit from paging that is considerably more resource efficient.

3.3 Atlas Design and Implementation

This section presents Atlas’s design. Like AIFM, Atlas requires programs to use smart pointers (*i.e.*, to implement barriers) and declare dereference scopes for objects (inspired by C++ weak pointers [6] and Folly RCU guards [5]). Objects are managed by Atlas’s hybrid data plane. Atlas can also take the same user-defined programming/offloading hints and object-level prefetching logic as required by AIFM. Atlas uses such hints in the object fetching path.

3.3.1 Overview

Inspired by the design of the Java heap [97], Atlas divides a page into *cards* to enable fine-grained profiling for accesses. For each page, Atlas builds a card access table (CAT), which is a bitmap where each set bit represents a card that has been accessed since the page was allocated or last swapped in. CATs for contiguous pages are allocated contiguously in a separate memory space. This design enables not only fine-grained access profiling, but also simple mapping from a virtual address to its CAT entry—this can be done with efficient bit-wise operations on the address. Each card represents 16 consecutive bytes, which provides a fine enough granularity as most objects are at least 16 bytes in our workloads.

Atlas maintains a 1-bit *path selector flag* (PSF) for each page, which works as an indicator of the data path for data access on the page. A **runtime** value indicates that data should be retrieved by the runtime at the object granularity (*i.e.*, runtime path). A **paging** value indicates that data should be paged in by the kernel (*i.e.*, paging path). Atlas updates the PSF of each page to **runtime** or **paging** at the moment the page is swapped out if its CAR (*i.e.*, the percentage of the set bits among all bits in a CAT) goes below or above a threshold (*i.e.*, 80% used in our evaluation, see §3.4.4).

Ingress. Atlas uses a *read barrier* that executes at each *smart pointer dereference*. The barrier first checks whether the accessed data is remote. In AIFM, this is done by using a bit in each pointer to encode the location of the referenced object—these pointers are updated once the objects they point to are swapped in or out. Atlas, however, cannot adopt this approach due to the use of the hybrid data plane—when data is *paged* out, Atlas cannot update any pointers. To solve the problem without incurring the cost of checking with the kernel at every read, Atlas leverages *hardware transaction memory* and, in particular, Intel’s TSX [58], to run a quick check—Atlas accesses the address in a hardware transaction, which aborts if the address is not on a mapped page.

Upon an abort, the barrier reads the PSF for the page to be accessed and determines which path (runtime *vs.* paging) the access should take. If the runtime path is taken, the object is moved (*i.e.*, address changed) to a local page on the compute server and its pointers are updated; otherwise, the page containing the object is swapped in as a whole and the address of the object remains the same (without requiring pointer updating).

Egress. Given that the majority of the object-fetching overhead comes from the need to find and evict cold objects, Atlas utilizes a single path, *i.e.*, paging, to swap out data. This approach achieves a sweet spot in balancing overhead and benefits—on one hand, it significantly reduces the compute resource usage for object fetching because of the elimination of maintaining an object-level LRU; on the other hand, given that object fetching gradually improves locality (by moving together objects accessed closely in time), the amount of useless data in each swap-out (and thus the I/O amplification) is reduced progressively during execution.

Another reason to not evict objects individually is that it can potentially *hurt locality*—after objects are fetched in, those that were scattered in remote memory but accessed together were moved into contiguous local space; however, these objects may not be evicted at the same time; evicting them individually would make them go to unrelated locations in

remote memory, disrupting established locality.

Synchronization. Allowing the two paths to co-exist in harmony requires overcoming the following three synchronization challenges: (1) *ingress synchronization* between object-in and page-in, (2) *egress synchronization* between object-in and page-out, and (3) *move synchronization* between object-in and evacuation. AIFM already solves the third problem with the declaration of dereference scopes, while the other two are unique challenges that we target in Atlas.

3.3.2 Synchronization of the Two Paths

Atlas builds its object fetching path upon the same two abstractions used by AIFM: the smart pointer (which is an extension of C++ smart pointer) and the dereference scope. This section elaborates on the synchronization mechanism between the object fetching path and the paging path.

```

1  class AtlasUniquePtr<T>{
2      struct AtlasMetadata{
3          unsigned long is_moving : 1;
4          unsigned long access : 1;
5          unsigned long reserve : 2;
6          unsigned long offload : 1;
7          unsigned long size : 12;
8          unsigned long addr : 47;
9      } metadata; // 64 bits
10     AtlasUniquePtr(T* obj);
11     T* get_raw();
12 }

```

Figure 3.2: Atlas unique pointer metadata.

Pointer Metadata. Before discussing our barrier logic, we first present the format of Atlas pointers, which are built on C++ smart pointers. Atlas uses two types of smart pointers: unique pointers (similar to `std::unique_ptr`) and shared pointers (similar to `std::shared_ptr`). Figure 3.2 shows the layout of an Atlas unique pointer. These fields are added for the purpose of synchronization and pointer updating.

Each such pointer has 64-bit metadata, in which 47 bits (`addr`) store the object’s raw pointer, 12 bits (`size`) record its size, 1 bit (`access`) represents whether the object has been accessed since the last evacuation (which will be used by the evacuator to group recently accessed objects, see §3.3.3), 1 bit (`offload`) indicates whether a function is being invoked on the object on the remote side, and 1 bit (`is_moving`) indicates whether the object is being moved (*e.g.*, due to evacuation); this bit will be used for synchronization between two threads trying to move the same object. The remaining 2 bits (`reserve`) are reserved for future use. Note that 12 bits can represent a size up to 4KB. Objects larger than that are placed in the huge-object space of the heap for which paging is the only option. `get_raw` retrieves the raw pointer from a smart pointer.

A shared pointer allows aliasing. Atlas treats the first shared pointer of an object as the main pointer. A shared pointer’s layout is similar to a unique pointer, except that it has an additional 8 bytes to chain all pointers—when the main pointer is being released, Atlas follows the chain to select a new main pointer. If an object is referenced by shared pointers, Atlas needs to update all of them (by following the chain).

Developers need to explicitly declare data types with smart pointers. Developers can access data with raw (regular C++) pointers by first retrieving such raw pointers from smart pointers. However, this can only be done within an explicitly declared dereference scope. Figure 3.3 illustrates an example of retrieving and manipulating data from Atlas smart pointers, confined by a dereference scope. As discussed in §2.1.2, dereference scopes synchronize with object migration tasks—once raw pointers are retrieved and actively used, their objects are not allowed to move, and vice versa. Atlas executes a `pre_scope_barrier` and a `post_scope_barrier` at the beginning and the end of the dereference scope, respectively.

Synchronization invariants. We present a set of high-level invariants that Atlas maintains to solve the three synchronization problems: (1) preventing an object from being fetched from the two paths simultaneously (object-in *vs.* page-in), (2) preventing pages containing

```

1  deref_scope (smart_ptr) {
2      pre_scope_barrier(smart_ptr); // Algorithm 1
3      Data * object = smart_ptr.get_raw();
4      /* Operations using the object */
5      ...
6      post_scope_barrier(smart_ptr); // Algorithm 2
7  }

```

Figure 3.3: Dereferencing an Atlas unique pointer in a deref scope.

objects that were just runtime-fetched from being immediately swapped out (object-in *vs.* page-out), and (3) preventing an object from being simultaneously runtime-fetched and moved by the evacuator (object-in *vs.* evacuation).

Invariant #1: Object-in vs. page-in. At any moment, all data on the same page must go through the same access path as guided by the page’s PSF. In other words, Atlas prohibits scenarios where certain requests are served by paging while others are served by the runtime for the same page. Given that Atlas changes PSF only at page-out (as opposed to setting it while the page is in local memory), such scenarios can never occur and this invariant is guaranteed by design.

Note that there is no issue if two threads fetch the same page from the paging path—the kernel’s swap system guarantees only one page can be mapped. Fetching the same object from two threads with the runtime path is not a concern either: it is a solved problem in the literature of moving garbage collectors [81] where pointer updating is used as a synchronization point and only one object is retained.

Invariant #2: Object-in vs. page-out. Since swap-out events can occur at any time with the runtime path uninformed, Atlas enforces that pages containing objects whose dereference scopes are actively executed cannot be swapped out. This is because if such pages are swapped out before their dereference scopes finish, these objects may be fetched back in immediately from the runtime path, requiring pointer updating. Pointer updating cannot be done when the raw pointers of these objects are active on the stack. As a result, these pages cannot be swapped out until none of their objects are executing their dereference scopes.

Algorithm 1: Atlas Pre-Scope Barrier (Simplified).

```
/* derefcnt > 0 precludes the page's swap-out */
1 atom_inc(find_page_meta(addr).drefcnt)
2 if not tsx_check_local(addr) then /* Remote object */
3   if take_runtime_path(addr) then /* Runtime path */
4     new_addr ← find_addr(addr, this.size)
5     /* Inc/dec the new/old page's derefcnt */
6     atom_inc(find_page_meta(new_addr).drefcnt)
7     atom_dec(find_page_meta(addr).drefcnt)
8     alloc_copy_update(addr, new_addr, this.size)
9     this.metadata.addr ← new_addr
10  else /* Paging path */
11    * (char*) addr
```

Algorithm 2: Atlas Post-Scope Barrier.

```
1 atom_dec(find_page_meta(this.addr).drefcnt)
```

Atlas achieves this by maintaining a per-page *deref count*, which is incremented when any object on the page enters a dereference scope and decremented when the scope finishes. Any page with a non-zero deref count is skipped when the kernel looks for swap-out victims. Note that this does not create much impact on performance because the pages whose objects are actively used are usually hot pages and unlikely to be selected as swap-out victims anyway.

One issue that may arise from this protection is a potential live lock on the object-fetching path: either an ill-defined large dereference scope or many active dereference scopes in a parallel application may potentially lead to too much data getting pinned in local memory, which may result in out-of-memory errors. To tackle this issue, Atlas monitors the pinned data and forces the flipping of their containing pages' PSFs (to use paging) upon memory pressure. Once these pages are swapped out, they will be paged in—this solves the problem as page-in does not need pointer updating.

Invariant #3: Dereference scope vs. evacuation. Evacuation threads may move an

object while another thread is executing the object’s dereference scope. This must not occur because evacuation requires pointer updating, which cannot be done when a dereference scope is being executed (and raw pointers are used). To this end, Atlas uses the page’s deref count to synchronize between evacuation threads and dereference scopes. A non-zero dereference count prevents the page from being evacuated.

Compared to AIFM, Atlas employs a slightly different definition of dereference scope. AIFM chose to decouple dereference scopes from the barrier—it allows one dereference scope to cover multiple smart pointer dereferences, serving as a coarse-grained fence between application threads and the evacuator. On the contrary, Atlas employs *fine-grained* dereference scopes, each of which is associated with one single smart pointer dereference. This choice was made based on our observation of frequent evacuations; using coarse-grained dereference scopes would require constant synchronizations between application and evacuation threads, leading to performance and latency impact. Fine-grained dereference scopes not only reduce the degree of blocking but also help alleviate potential live locks. Although a finer granularity increases barrier overhead, this overhead is often amortized by a large number of raw pointer accesses and computation within each scope. A detailed overhead analysis can be found in §3.4.2 and §3.4.4.

With the invariants discussed above, we proceed to presenting our barrier logic, which is shown in Algorithm 1 and Algorithm 2. As illustrated in Figure 3.3, Atlas executes Algorithm 1 and Algorithm 2 at the beginning and the end of a dereference scope, respectively.

Pre-scope barrier. Atlas first atomically increments the deref count for the page containing the object (Line 1). This indicates that the page has an object whose dereference scope is being executed, preventing the paging system from swapping out the page (*i.e.*, Invariant #2). This step must be done before the barrier starts to guarantee that (1) if the page is local, it cannot be swapped out from this point on, or (2) if the object is remote, once it is fetched in, its containing page cannot be swapped out.

Atlas uses Intel’s TSX [59] to efficiently check if the address `addr` is local. Atlas starts an RTM transaction, which contains nothing but a dereference of the object. If the object’s containing page is unmapped, the RTM transaction will abort with a special status captured by Atlas, which verifies the status by checking with the kernel. This hardware-based check is $\sim 14\times$ faster than a purely software-based approach that relies on a system call that walks the page table and checks whether the page is local based on its PTE. A `true` value (*i.e.*, the object is local) returned by TSX directs the execution to exit the barrier immediately. Otherwise, Atlas checks the PSF corresponding to the address (Line 3) to decide whether this access should take the runtime (Lines 4-9) or the paging path (Line 11).

Using TSX to check object location may introduce false positives—a transaction may abort even if data is local. Since such cases are rare (*e.g.*, less than 1/10000 in our experiments), Atlas takes an optimistic approach to handle them. Upon a TSX abort, Atlas sends an RDMA read to access the remote object and simultaneously issues a page table walk to verify the object’s location. If the verification fails (indicating the object is local), the fetched object is discarded. This approach introduces only a negligible overhead (*i.e.*, a small number of unnecessary RDMA reads).

Runtime path. `take_runtime_path` in Algorithm 1 checks the PSF of the page corresponding to `addr` and returns `true` if the PSF is `runtime`, indicating that object fetching should be performed. For ease of presentation, Algorithm 1 is significantly simplified to *not* show details of how to synchronize between threads to guarantee the absence of race condition when multiple threads fetching the same object. Atlas first finds a new address to which the object will be moved (Line 4). Since this address is on a new page, before moving the object, the deref count of the new page must be incremented (Line 5) to ensure that from this point on, the new page cannot be swapped out until the dereference scope finishes (*i.e.*, Invariant #2). The barrier also needs to decrement the deref count of the old page (Line 6) that was incremented earlier in Line 1.

Next, Atlas fetches the object by allocating a new object of the same size (using our log-structured allocator discussed in §3.3.3), copying the object’s data into the new object, and updates its pointers (Line 7). Atlas subsequently changes the `addr` field of the pointer to the new address (Line 8). Pointer updating is done by retrieving the object’s pointer from its header and updating their addresses, in a way similar to how it is done in AIFM. If it is a shared pointer, all other pointers will be retrieved from the main one and updated accordingly. The object’s `is_moving` field is used to synchronize between pointer updating events performed by multiple threads. The synchronization details are omitted for simplicity. After the object is moved to a local page, future accesses to the object will follow the PSF of the new page.

Paging path. The paging path simply touches the object (Line 11) to ensure that the page fault handling is *completed* after the execution passes this line.

Post-scope barrier. The post-scope barrier has much simpler logic, as shown in Algorithm 2. All it needs to do is to atomically decrement the page’s deref count, indicating the finishing of the dereference scope. When its deref count becomes zero, this page is subject to swap-out again (*i.e.*, Invariant #2).

3.3.3 Memory Management

Atlas’s heap is composed of a *normal-object* space, a *huge-object* space, a *metadata* space, and an *offload* space. Atlas manages the normal-object space via a log-structured allocator [103, 106] and maintains a background evacuator to reduce fragmentation by compacting live objects. Atlas does not handle huge objects that cannot fit into a page, placing them into the huge-object space and delegating their management to the kernel directly since they are too large to benefit from object-level management. Metadata such as CATs are accessed by both the runtime and paging system, and hence, it is shared between the user and kernel

space. The offload space stores objects whose functions can be offloaded to the remote side. We will discuss it shortly.

Object allocation. The log-structured allocator maintains thread-local allocation buffers (TLAB) to reduce the global lock contention during parallel object allocation. The TLAB is managed at the granularity of log segment which is aligned with a page to guaranteed that no object can go cross the page boundary. Atlas allocates objects contiguously on the TLAB as prior research [119, 126] shows that objects allocated close in time exhibit similar usage patterns. In doing so, objects with temporal proximity are naturally grouped into the same log segment (page), enhancing locality.

Metadata allocation. Metadata such as dereference counters and card tables is allocated in a dedicated metadata space. Atlas maintains a card table for each page to record the object access information. Each card table is a bitmap where each bit represents a consecutive range of 16 bytes. Our experiments show that the sizes of most objects are larger than 8 bytes, making 16 bytes a natural choice for the card size. Each card table is allocated and initialized during the allocation of a log segment. It is freed along with the log segment. The space needed by the card tables is $1/128$ of the total memory. In summary, the space overhead is less than 2%.

Object evacuation. The log-structure allocator [106] supports defragmentation via a copying-based evacuator, a technique widely used in modern garbage collectors [40]. In Atlas, we extend the evacuator to improve the temporal locality of pages by grouping hot objects into contiguous log segments (pages) during the evacuation. The evacuator runs concurrently with the application to reduce fragmentation.

The evacuator periodically scans log segments and evacuates a log segment with a high garbage ratio by copying its live objects to a newly allocated target segment. As a result, the target segment is free of fragmentation, and the source log segment can be freed right

away. When moving an object, the evacuator maintains its corresponding card table values, *i.e.*, if the object was recently accessed on the source page, the evacuator marks its card bit on the target page during evacuation. Furthermore, Atlas improves evacuation efficiency by prioritizing log segments in local memory and delaying the processing of remote log segments until they are accessed or the free space runs out [121].

The Atlas runtime tracks whether an object has been accessed since the last evacuation via the `access` bit in the smart pointer (see Figure 3.2). This bit is set by the read barrier when the object is dereferenced and cleared by the evacuator at the end of each evacuation. The evacuator segregates objects that have been accessed since the last evacuation into a set of contiguously allocated log segments. We found this approach to be particularly effective in improving temporal locality for real-world workloads with *skewness* (*e.g.*, 90% of accesses hit 10% objects). The `access` bit allows Atlas to distinguish hot and cold objects in such workloads, leading to a substantial performance boost. Note that this operation is significantly more efficient than maintaining an object-level LRU for eviction. As opposed to ranking objects based on hotness, Atlas’s `access` bit simply serves as an evacuation location indicator. Its functionality is similar to CAT but used differently; CAT is read and cleared by the kernel at page eviction while the `access` bit is read and cleared by the runtime at evacuation.

Computation offloading. As shown in many existing remote memory systems, such as Semeru [120], Mako [81], AIFM [103], and Mira [49], offloading memory-intensive operations to the remote side can effectively reduce the data movement overhead. A unique challenge for Atlas is how to enable offloading when paging is used. Under paging, remote memory is managed as a swap partition of a set of swap slots. These slots are agnostic about the remote server’s memory addresses. Pointer addresses contained in a page are with respect to the compute server while the page can reside at a completely different address on the remote server. This address mismatch precludes the correct execution of a function on an

object directly on the remote server.

To solve the problem, Atlas uses an approach that is similar to Semeru [120]—we reserve a dedicated offload space in the heap. Developers need to explicitly define remoteable data structures and functions (which are similar to those in AIFM). Objects registered as *remotable* are all allocated into this space. Pages in this space have guaranteed virtual address alignment between the compute and remote servers—we modify the paging system to ensure that a page at a virtual address *A* on the compute server is guaranteed to be still at address *A* on the remote server when evicted. Atlas requires users to guarantee a remotable data structure cannot reference a non-remotable object. This property ensures address consistency when a function is called remotely.

The offload space is an *object-in, page-out* space, which allows objects to be fetched only through the runtime. This is due to the need to synchronize between the servers for safe remote execution. When a remote function is being invoked on an object, the `offload` field in its smart pointer is used for synchronization—the runtime can not fetch the object until the remote function is finished (and the `offload` bit is cleared). Remotable objects can only be fetched into the offload space to ensure the above-stated properties.

3.4 Evaluation

3.4.1 Setup and Methodology

We wrote 7,675 lines of C/C++ code to implement Atlas’s runtime library, and added support in the Linux kernel (version 5.14-rc5) for page management (*e.g.*, path synchronization). We ran experiments with one compute server and one memory server connected by a 200 Gbps Infiniband switch. Each server has 2 Intel Xeon Gold 6342 CPUs (24 physical cores each), 256 GB of memory, and a 100 Gbps Mellanox ConnectX-5 InfiniBand adapter. All evaluated systems ran on Ubuntu 18.04. We configured the servers following common practice for low latency [98], disabling Turbo Boost, CPU frequency scaling, and transparent huge pages.

Application	Dataset	Size	Characteristics
Memcached CacheLib [7] (MCD-CL)	Meta CacheLib [20]	50M records	Skewness with churn
Memcached Uniform (MCD-U)	Synthetic [32]	50M records	Random access
GraphOne PageRank [65] (GPR)	Twitter 2010 [67]	1.5B Edges, 41.7M Vertices	Evolving graph
Aspen TriangleCount [35] (ATC)	Friendster [130]	1.8B Edges, 65.6M Vertices	Evolving graph
Metis Word Count [83] (MWC)	News Crawl [128]	5.1GB	Phase-changing
Metis PageViewCount (MPVC)	Wikipedia English [8]	15GB	Phase-changing
DataFrame [87] (DF)	NYC Taxi [4]	16 GB	Phase-changing with offloading
Web Service [103] (WS)	Synthetic [103]	10GB hashmap, 16GB array	Mixed patterns with offloading

Table 3.1: Applications used for Atlas evaluation.

Baselines. Atlas was implemented based on Fastswap and AIFM. For the paging path, Atlas uses unmodified Fastswap with added tasks of profiling and synchronization. For the runtime path, Atlas uses AIFM’s ingress algorithm and paging at egress. For evaluation, we used AIFM [103] and Fastswap [15] as our baselines for object fetching and paging, respectively. For Fastswap, we ran the original applications to avoid unnecessary runtime overhead. For AIFM, we used the performance-tuned versions of applications, where all optimizations were enabled including per-thread access pattern tracking, object hotness tracking, and non-temporal programming hints [103]. We turned off offloading when evaluating throughput and latency, leaving its evaluation to §3.4.4.

Workloads. As shown in Table 3.1, we evaluated six real-world applications and two synthetic applications, including Metis [83]—an optimized MapReduce framework for multi-core architectures, Aspen [35]—a purely functional tree-based graph processing framework, GraphOne [65]—a data store for real-time analytics on evolving graphs, as well as Memcached [7]—an in-memory key-value store. We ran Memcached with two different workloads: a real-world workload (MCD-CL) that comes from Meta’s cache system CacheLib [20] and a synthetic workload (MCD-U) generated by YCSB [32] that follows a uniform distribution. We also employed two synthetic applications developed by AIFM’s authors to compare Atlas and AIFM. These applications include one batch application, DataFrame [87], and one latency-critical application, WebService.

Covering a wide spectrum of domains and memory access patterns (*i.e.*, sequential, random, skewed, and mixed patterns), these applications can be divided into four categories:

First, both Memcached workloads exhibit random access patterns, leading to significant I/O amplification under paging. The real-world workload MCD-CL has a high level of skewness with *churn* behaviors. *Churn* refers to the phenomenon that hot data in the working set changes rapidly over time. On the contrary, the synthetic workload MCD-U demonstrates completely random behaviors, with no skewness and hot data. As a result, MCD-CL is more amenable to Atlas’s dynamic locality improvement than MCD-U.

Second, GraphOne and Aspen are evolving graph systems, which are representatives of applications that perform analytics over frequently updated datasets. GraphOne uses adjacency lists and edge lists to store an input graph while Aspen utilizes compressed purely-functional trees to store a graph, which supports a higher update rate. The working sets of these applications change continuously. Their accesses are very complex: the first stage builds the graph in memory, exhibiting a random pattern. The second stage runs iterative algorithms where the first iteration does not have locality and thus performs random accesses; the subsequent iterations would enjoy better locality if it runs on Atlas, which dynamically improves the locality during the first iteration. However, updates to the input graph disrupt the locality and hence there can also be many random accesses in the middle of the iterations. We used these two graph frameworks to evaluate how well Atlas can dynamically adjust the data layout and improve locality.

Third, Metis (MapReduce) and DataFrame represent bulk data processing systems with clear phase-changing behaviors (discussed in §3.2). These workloads are used to evaluate whether Atlas can accurately recognize access patterns and switch to the proper data path. DataFrame is additionally used to evaluate compute offloading due to its memory-intensive operations (§3.4.4).

Finally, WebService is an interactive web application exhibiting mixed access patterns, from random, pointer-chasing, to sequential accesses.

For Atlas to run these applications, we modified 263 lines of code for Metis, 278 lines for Aspen, 219 lines for GraphOne, and 391 lines for Memcached; the additional code was used

to declare smart pointers and dereference scopes. It took one developer a few hours to port each program.

Memory setup. Each application was run with five local memory configurations: 13%, 25%, 50%, 75% and 100%, each representing a specific percentage of an application’s working set that can fit into local memory. These configurations were enforced using `cgroup`. The first four configurations were employed to evaluate the performance of the three systems when using different amounts of remote memory, while the 100% (all local memory) configuration was used to assess the runtime overhead of Atlas and AIFM, introduced by the barriers (for smart pointer dereferencing), dereference trace recording (for object-level prefetching), and evacuation (for defragmentation), as well as other bookkeeping overheads; see Table 3.2 for more details.

3.4.2 Throughput

We first measured the throughput of the applications with varying local memory ratios. Overall, Atlas outperforms Fastswap and AIFM, respectively, by 3.2× and 1.5×, over the eight real-world applications using remote memory (from 13% to 75% local memory). When running locally (100% local memory), Atlas and AIFM incur an overall overhead of 19.1% and 14.0%, respectively, of which 10.2% and 2.3% are from the barriers. This section reports the overall performance and runtime overhead. We show a detailed overhead breakdown in §3.4.4.

MCD-CL and MCD-U. Both workloads were configured with the same operation ratios, *i.e.*, 87.4% get and 12.6% set. As shown in Figure 3.4(a), for a highly-skewed workload like MCD-CL, both Atlas and AIFM outperform Fastswap (by 6.4× and 3.2×, respectively). The performance difference comes primarily from the reduced I/O amplification—Fastswap fetches 26× and 30× more data than Atlas and AIFM, respectively, resulting in wasted

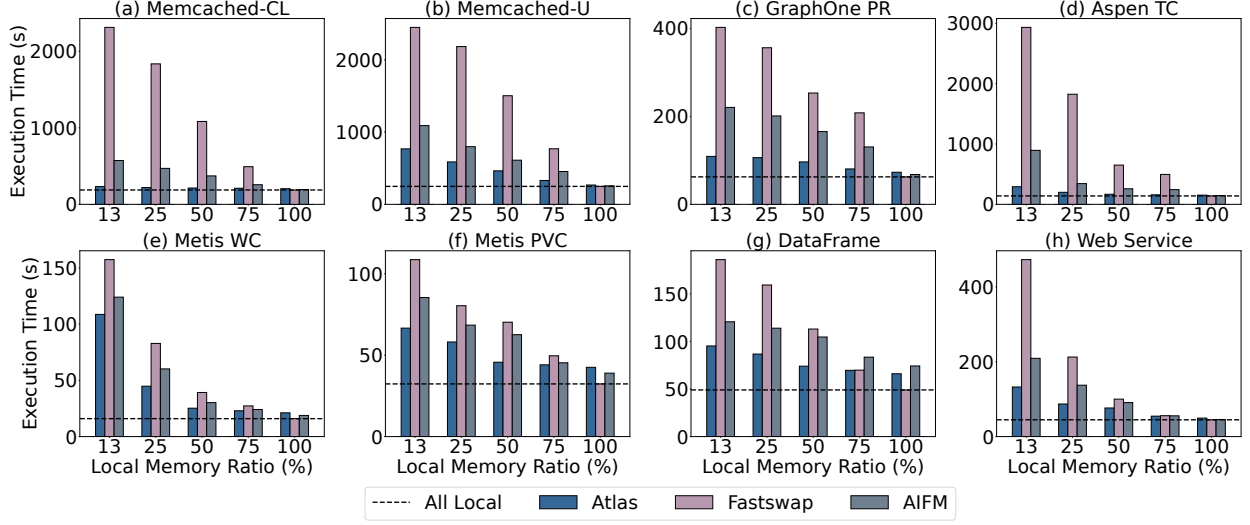


Figure 3.4: Throughput comparison between Atlas, Fastswap and AIFM with varying local memory ratios. “All Local” lines represent the performance of unmodified applications under 100% local memory.

memory (for storing unused data) and significantly more swaps. Under 100% local memory, Atlas and AIFM introduce an overall overhead of 9.0% and 3.2%, respectively, compared to Fastswap. The primary source of the overhead is the barriers, taking 6.2% and 1.5% of the execution time, respectively. Given that Memcached spends a substantial portion of its execution on communication, the barrier overhead, which is associated with the in-memory processing, is insignificant.

Compared to AIFM, Atlas further improves the performance by 1.2 $\times$, 1.8 $\times$, 2.2 $\times$, 2.5 $\times$, under the four different memory configurations (75%, 50%, 25%, and 13%). This improvement stems from a much higher eviction throughput (on average 4.6 $\times$ higher) in Atlas due to the elimination of object eviction. In addition, Atlas’s concurrent evacuator (§3.3.3) improves the temporal locality by segregating hot objects into contiguous pages, leading to an overall of 18% more accesses that go through the paging path (§3.4.4). This result was achieved when AIFM used 20 eviction threads while Atlas only used one single swap-out thread in the paging path. MCD-U performs random accesses with no hot data, hindering opportunities for Atlas to improve locality. Hence, the usefulness of the hybrid data plane is

limited. However, Atlas still outperforms AIFM by up to 1.4 $\times$ due to more efficient eviction, as shown in Figure 3.4(b).

GPR and ATC. To execute an evolving graph engine, we divided the input datasets [67] into three batches, which are incrementally fed to the graph engine. For each batch, the graph engine conducts the following three steps: load the updates, update the graph, and execute the analytics.

As Figure 3.4(c) shows, in the presence of remote memory, Atlas outperforms AIFM and Fastswap by an average of 1.8 $\times$ and 3.1 $\times$, respectively, on GPR. As stated earlier, graph updating and the first iteration of analytics exhibit random access patterns. As such, GPR’s throughput under AIFM is 1.7 $\times$ higher than under Fastswap. For Atlas, when the analytics starts, objects are accessed and reordered by the object fetching in the first few iterations; in the subsequent iterations, pages storing edge objects are switched to using the paging path due to the gradually established locality. As a result, up to 82% of pages have their PSFs changed during the execution (from object fetching to paging), as demonstrated in Figure 3.7(b). This improves the analytics throughput.

ATC’s computation stages and access patterns are both similar to those of GPR. For ATC, the trees storing the graph data are dynamically reorganized by Atlas’s runtime path, leading to $\sim 38\%$ of pages changing their PSFs (from object fetching to paging). In addition, evacuation improves locality by segregating hot objects from these trees into a few pages, reducing remote memory accesses by 24%. As demonstrated in Figure 3.4(d), ATC’s overall throughput is 2.0 $\times$ higher under Atlas than under AIFM.

When running on 100% local memory, Atlas’s barrier overheads for both GPR and ATC are modest, 8.2% and 4.3%, due to the high ratio between raw pointer accesses and smart pointer dereferences. Oftentimes, one object dereference (*e.g.*, obtaining a vertex that contains a series of edges) is followed by dozens of raw pointer accesses (*e.g.*, to individual edges). Each dereference scope contains an average of 21 raw pointer accesses. In addition,

for ATC, the barrier overhead is further diluted due to its higher computation and memory access costs (from poor spatial locality).

MWC and MPVC. Figure 3.4(e) and (f) respectively show the performance of MWC and MPVC. As discussed in §3.2, MPVC exhibits a two-phase behavior that can benefit from adaptive path switching, leading to a 1.2× and 1.4× improvement, compared with AIFM and Fastswap, respectively. MWC has a similar two-phase behavior with MPVC but exhibits more random accesses in its map phase, resulting in almost no page that can be flipped to `paging`. Compared to AIFM and Fastswap, MWC has 1.2× and 1.5× performance improvement, respectively.

For these two applications, the runtime overhead is relatively high—32.0% (Atlas) and 19.2% (AIFM), under 100% local memory. These two Metis workloads are both memory-intensive—they keep scanning data with high parallelism, leading to both high barrier overhead and profiling overhead (*e.g.*, for card profiling and access trace recording, see §3.4.4). Atlas’s barrier overhead reaches up to 16.1% and 17.4% for MPVC and MWC, respectively, which are about 4× higher than that of AIFM.

DF. DF is a table-structured in-memory data structure with hundreds of columns and millions of rows, popularized in Pandas [94]. Users can slice data in different ways and run various statistics. As Figure 3.4(g) shows, Atlas outperforms AIFM by 1.2~1.4× in the four remote-memory settings. We ran a client, developed by the AIFM authors, to conduct a series of *Copy* and *Shuffle* operations on *DF*. Similarly to Metis, *DF* demonstrates clear phase-changing behaviors when processing different operations—a *Copy* operation copies data from a column, exhibiting excellent spatial locality and a clear sequential pattern, while a *Shuffle* operation reorders rows for each column, exhibiting random patterns. Atlas achieves superior performance to AIFM and Fastswap, due to its adaptive access path selection.

AIFM suffers a higher runtime overhead (51.4%) compared to Atlas (34.7%) despite having a lighter barrier. The reason is that AIFM maintains a remote vector on the memory server for every DataFrame vector to support the eviction of individual objects with varied sizes. During the execution, DataFrame vectors keep getting allocated and resized. As a result, the remote data structure also needs to be frequently resized to maintain a valid mapping from local objects to their remote memory locations. Resizing is a heavy operation as it requires allocating memory and moving all existing objects. Therefore, it becomes a major source of overhead, which can take two-thirds of the runtime overhead under 100% local memory. On the other hand, under Atlas, eviction is handled by the Linux kernel at a fixed page size and there is no need to maintain any remote data structures. Note that frequent resizing of data structures was not observed in other applications. For example, for WS, the hash table array is allocated at the start of the application and its size remains fixed throughout the execution.

WS. WS is implemented by AIFM’s authors to simulate a distributed workload. Each client (thread) sends 32 requests to look up keys in an in-memory hash table and fetches a single 8KB element from an array. This element is then encrypted with Crypto++ [9] and compressed using Snappy [43] before being sent back to the client. We use a 26GB dataset for the evaluation, which is consistent with the dataset used in AIFM [103]. Client requests are generated by following a Zipfian distribution.

As Figure 3.4(h) shows, compared to AIFM, Atlas improves WS’ performance by an average of 1.3× with remote memory. This is due to an extremely large number of objects on the LRU list that must be analyzed by AIFM. AIFM’s performance degradation is primarily due to the compute resource contention between application and evacuation threads (discussed in §3.2), making it hard for evacuation threads to quickly identify and evict cold objects. Consequently, AIFM ends up evicting arbitrary objects to reclaim memory, resulting in data thrashing. By using paging for eviction, Atlas improves the eviction throughput by 5.8×,

lifting data eviction efficiency to 5.9 cycles/byte, which is 7.4 $\times$ higher than that of AIFM (43.7 cycles/byte).

Atlas and AIFM have relatively low overhead for WS due to the coarse-grained data fetching (8KB element) and the subsequent compute-intensive encryption. As a result, Atlas and AIFM introduce a 10.1% and 1.9% runtime overhead under 100% local memory, respectively.

3.4.3 Latency

This section evaluates the latency distribution using the two latency-critical applications: WS and MCD-CL. The 25% local memory ratio was used in these experiments.

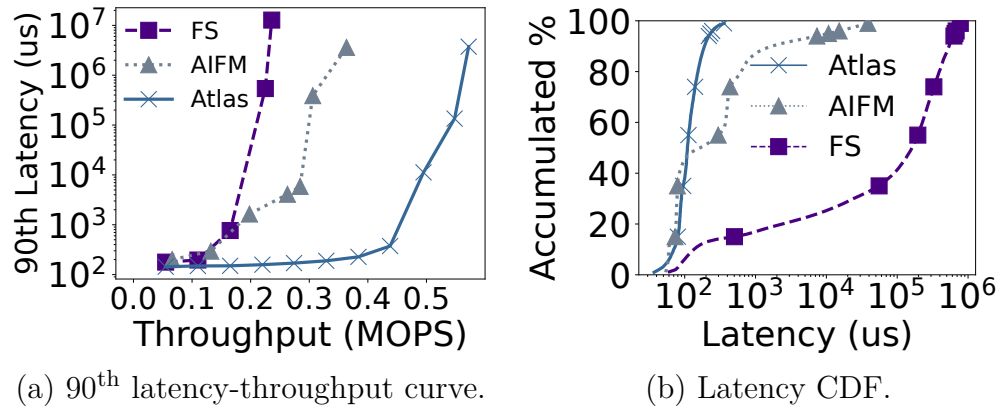


Figure 3.5: (a) 90th latency as a function of throughput; (b) Latency CDF under 0.23 MOPS offered throughput. FS stands for Fastswap.

Web Service (WS). Figure 3.5(a) compares the tail latency among the three systems. Fastswap’s tail latency rapidly grows due to page thrashing caused by severe access amplification. AIFM reduces amplification so that requests are less blocked by eviction. Despite the reduced amplification, AIFM still has to rank and evict individual key-value pairs, and hence the system saturates at 0.36 MOPS.

Atlas fetches individual key-value pairs initially via the runtime path and places those pairs which belong to the same request together on the same page (because these KV pairs

are accessed close in time). As the execution progresses, Atlas switches to paging that can load multiple key-values pairs at the same time. Meanwhile, page-level eviction continuously offers a much higher eviction throughput so that it never blocks swap-ins. As a result, Atlas’s tail latency stays low until 0.45 MOPS and can finally reach a peak throughput of 0.57 MOPS. As shown in Figure 3.5(b), the latencies of AIFM and Atlas are comparable until the 50th percentile, where the application starts accessing many remote objects leading to increased object management overhead. On the contrary, due to the optimized data layout which enables the efficient use of paging, Atlas experiences fewer remote accesses.

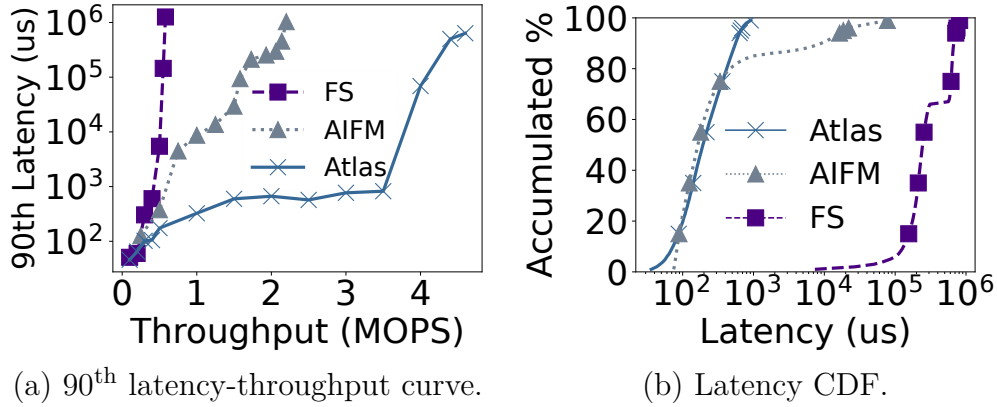


Figure 3.6: (a) 90th latency as a function of throughput; (b) Latency CDF under 1 MOPS offered throughput. FS stands for Fastswap.

MCD-CL. Memcached CacheLib is similar to Web Service as they both access key-value pairs from a hash table. The difference is that every request key in MDC-CL follows a Zipfian distribution, as opposed to accessing key-value pairs always in groups of 32. Figure 3.6 compares the tail latency among the three systems. It is clear that Atlas outperforms the other two systems. In addition to the same reasons explained above, MCD-CL is a skewed workload and hence a substantial portion (40%) of the improvement comes from the evacuation that groups hot objects in contiguous pages, making these pages amenable to paging.

3.4.4 Performance Drill Down

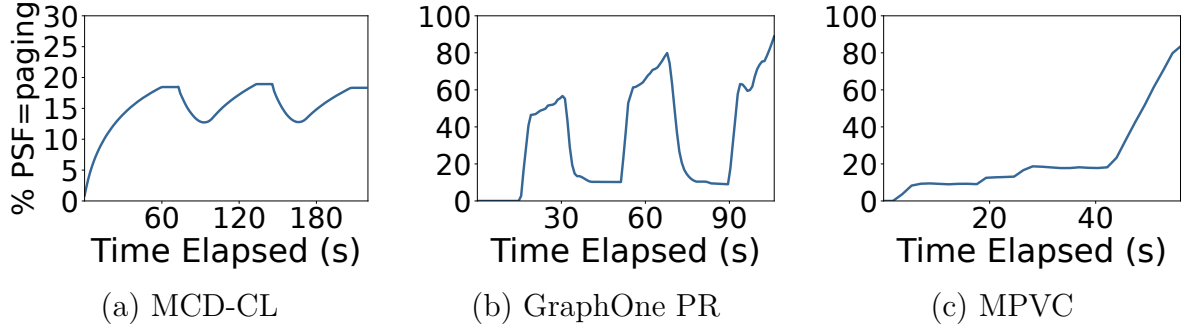


Figure 3.7: The percentage of pages with PSF=`paging` in the memory footprint changes with the elapsed execution time.

Adaptive path switching. To understand the effectiveness of Atlas’s adaptive path switching, we measured the percentage of the pages whose PSF is `paging` during the execution. Figure 3.7 demonstrates how this percentage changes during the execution for three applications: Memcached CacheLib (MCD-CL), GraphOne Pagerank (GPR) and Metis PageViewCount (MPVC). As Figure 3.7(a) shows, the number of pages that go through the paging path rises and falls over the time due to the *churn* behavior in MCD-CL discussed in §3.4.1. Since the workload is highly skewed, most accesses fall on a small number of hot objects, which stay in local memory and are moved into contiguous pages (with a high CAR) until the hot spot shifts.

As discussed in §3.4.1, the execution of GPR has experienced three batches of updates to the input graph, each of which contains two steps: graph building and analytics. During graph building, applying edge-level updates exhibits random access patterns, which can disrupt locality and leave many pages with a low CAR; these pages would have to go through the object fetching path. However, the subsequent analytics (like PageRank) runs multiple iterations; Atlas can quickly improve locality in the first few iterations, making pages turn their PSF to `paging` in subsequent iterations. This pattern can be clearly seen in Figure 3.7(b).

MPVC has a clear two-phase behavior (see Figure 3.1(a)) which can be accurately recog-

nized by Atlas —the number of pages that go through the paging path increases dramatically as the phase change is detected by Atlas (shown in Figure 3.7(c)). To understand the individual contributions of object fetching and evacuation to the locality, we disabled the `access` bit tracking and let the evacuator move live objects without guidance. This reduces the overall percentage of pages that go through paging by 4% on average.

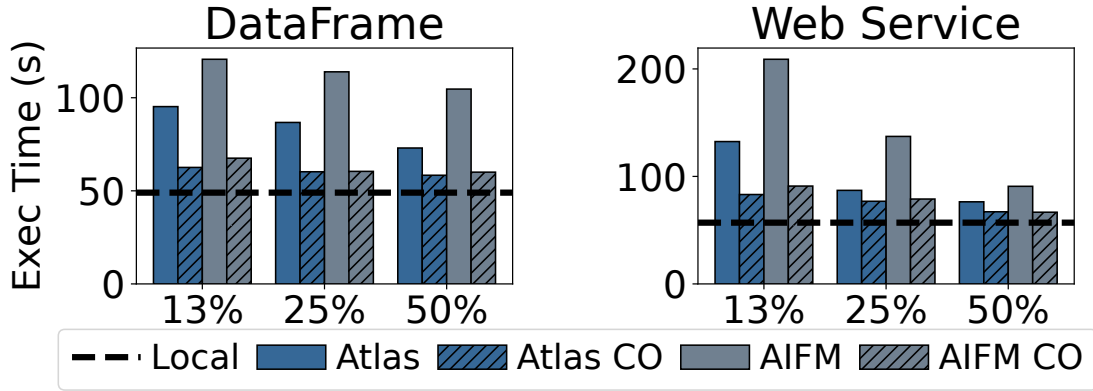


Figure 3.8: Throughput comparisons of DataFrame (DF) and Web Service (WS) when Atlas and AIFM enable compute offloading. CO stands for variants with compute offloading.

Computation offloading. We compared the offloading performance between Atlas and AIFM using DF and WS. Figure 3.8 shows the results of Atlas and AIFM with and without offloading. 18 cores were reserved on the remote side for both Atlas and AIFM, which is consistent with the offloading settings used by AIFM [103]. For DF, we offloaded the memory-intensive operations, *i.e.*, `Copy` and `Shuffle`, to the remote side. For WS, we offloaded the heavyweight array processing (on the 16GB data array). Compared to the setting where offloading is disabled (Figure 3.4 (g) and (h)), the throughputs of Atlas and AIFM are both dramatically improved (by up to 1.5× and 1.9× for DF, and 1.6× and 2.3× for WS, respectively), due to reduced remote accesses and data movement. On the other hand, Atlas and AIFM achieve comparable performance. This is because Atlas focuses on fetching efficiency; offloading reduces the need for fetching, making Atlas’s benefit less significant.

Runtime overhead analysis. To understand the performance penalty introduced by the runtime of Atlas and AIFM, we break down and compare the runtime overhead by sources. When running with all local memory, the runtime overhead of Atlas and AIFM can be divided into five major components, listed in Table 3.2. Note that the overhead reported here represents the **worst-case scenario** for Atlas when compared against AIFM. When there is remote memory, part of Atlas’s runtime overhead can be eliminated by switching to the paging path—dereference trace profiling is not used for paging as its goal is to analyze dereference traces for prefetching objects. Meanwhile, AIFM incurs more profiling overheads that do not exist under the all local memory setting, such as maintaining the object-level LRU for eviction.

Sources of overhead	Functionality	Affected systems
Barrier (Dereferencing)	Correctness guarantee, such as location check & synchronization	Atlas and AIFM
Card Profiling	Offering data path switching hints.	Atlas
Dereference Trace Profiling	Offering object-level prefetching hints	Atlas and AIFM
Evacuation	Defragmentation	Atlas and AIFM
Remote Data Structure Management	Managing object-level eviction	AIFM

Table 3.2: Major runtime overheads, operations involved in each type, and their affected systems.

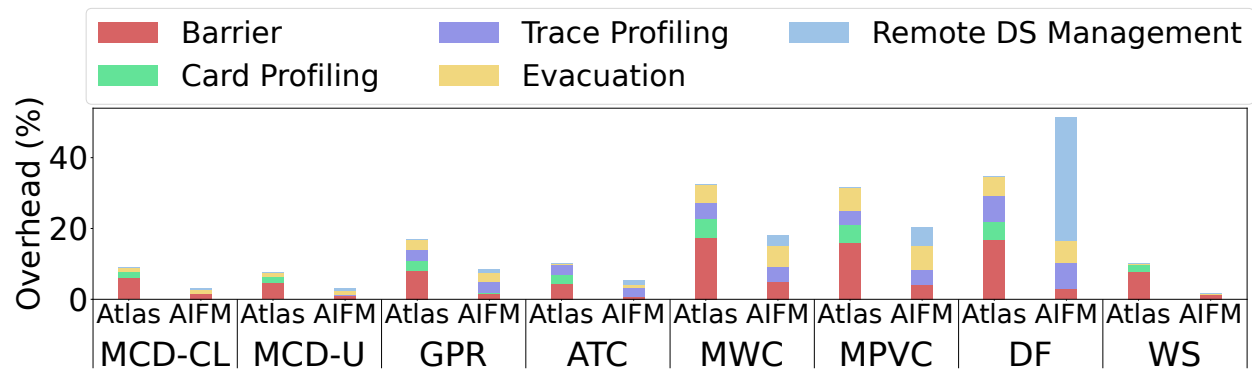


Figure 3.9: Runtime overhead breakdown: overhead is calculated as the ratio between the extra execution time introduced and the execution time under 100% local memory.

As shown in Figure 3.9, compared to Fastswap, the extra tasks in Atlas incur a runtime overhead of 7.7-34.7%, while AIFM’s overhead is 1.9-51.4%. The overall overheads of the two systems are 19.1% and 14%, respectively. The primary source of overhead for both systems is the barrier (except for DF with AIFM, for which the reasons are explained in §3.4.2). Specifically, the Atlas barrier accounts for half of the total overhead ($\sim 10\%$), and its cost is $4.4\times$ of that of AIFM. Note that this overhead correlates with an application’s memory access behavior: the most memory-intensive applications suffer the heaviest barrier overhead (MWC, MPVC, DF).

Although Atlas uses a heavier barrier, it underperforms AIFM by *only 4% under 100% local memory*. The reason is three-fold: (1) the barrier overhead is effectively amortized across the computation and raw pointer accesses (§3.4.2); (2) AIFM’s use of coarse-grained dereference scopes leads to higher synchronization costs than Atlas; and (3) there are other operations that also contribute to the runtime overhead. Since the first item has been discussed earlier in this section, here we elaborate on the second and third items.

The barrier conducts two basic tasks, object location checking and synchronization. For location checking, Atlas has a much higher overhead than AIFM due to the use of TSX to detect an object’s location whereas AIFM checks a bit on each reference. However, for synchronization, AIFM’s coarse-grained dereference scopes incur a higher cost, which effectively reduces the performance gap between the barriers of the two systems. After selecting the victim segments, AIFM’s evacuator must wait until all application threads exit their dereference scopes to avoid compacting objects being accessed through raw pointers. This design does not work well for big data applications with high object allocation rates, such as MWC, MPVC and Memcached. On the contrary, Atlas’s fine-grained dereference scope design enables evacuation threads to skip the segments (each aligned to a page in Atlas) whose *deref count* is non-zero (indicating they are being used in active dereference scopes) instead of blocking the whole evacuation, leading to significantly reduced synchronization efforts. In fact, Atlas’s CPU yield rate caused by synchronization is *an order of magnitude*

lower than that of AIFM due to our non-blocking design.

Another major source of overhead is the dereference tracing (to provide prefetching hints), accounting for 14% and 19% of the total overhead for Atlas and AIFM, respectively. Among our applications, DF, MWC, MPVC and GPR use array data structures which are amenable to prefetching. As a result, there is a relatively high tracking overhead (accounting for 34% overhead on average) for both Atlas and AIFM. Other applications such as WS and Memcached use hash maps and small objects as their data structures, which are not as amenable to prefetching as arrays. Hence, for most of their memory accesses, the locations are not tracked and their tracing overhead is much lower. Note that with remote memory, the dereference tracing overhead is significantly lower under Atlas than under AIFM because a large amount of data (*e.g.*, up to 82% for GPR) goes through the paging path, which utilizes the lightweight page-level prefetcher.

CAR threshold. Figure 3.10 shows the influence of CAR threshold on the throughput of three applications. Picking the right CAR threshold is a tradeoff between fetching efficiency and resource waste. We used 80% as the CAR threshold for flipping PSF in our evaluation. A higher CAR is often too conservative. For example, in the case of MCD-CL, when the threshold is set to 100%, we observed that few pages can be flipped to **paging**. Therefore, most remote objects still have to be fetched individually instead of fetched in batches with page faults, leading to a 25% decrease in throughput. On the contrary, a lower CAR may result in premature use of paging, leading to I/O amplification. As shown, the best performance is achieved when the threshold is between 80% and 90%. As such, we used the lower bound 80% based on the observation that the bandwidth of a modern network such as InfiniBand [95] is already high and will only become higher in the future, making it possible to transfer (slightly) more data with little overhead.

Hotness tracking. Atlas uses an **access** bit on each smart pointer to segregate hot and cold objects during evacuation, offering benefits to workloads that exhibit skewness. We

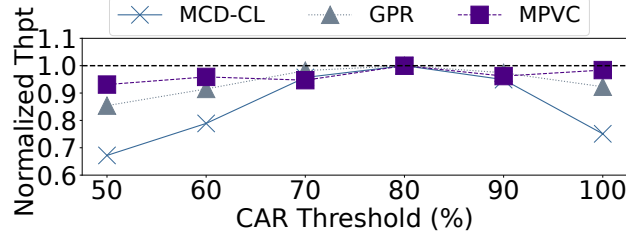


Figure 3.10: Sensitivity of the CAR threshold.

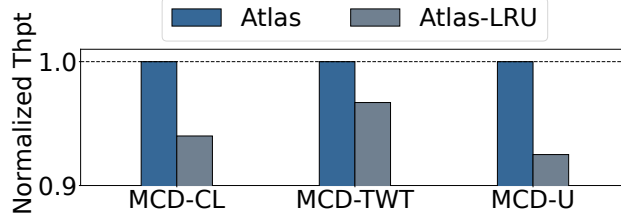


Figure 3.11: Normalized throughput of Memcached workloads running on Atlas and Atlas-LRU under 25% local memory.

evaluated the effectiveness of Atlas’s `access` bit with three skewed workloads, *i.e.*, highly-skewed (Meta, MCD-CL) [2], moderately-skewed (Twitter, MCD-TWT) [?] and uniform without skewnewss (MCD-U) [32]. We compared Atlas with a baseline (Atlas-LRU) equipped with an LRU-like policy from CacheLib [20], which represents a more accurate approach to identifying hotness.

As shown in Figure 3.11, Atlas’s single-bit design outperforms the LRU-like design by 7.5%, 3.3% and 6.0%, respectively. The LRU-like policy trades compute resources for accuracy by maintaining the logical ordering of objects via a linked list. Each dereference triggers a promotion that moves the object to the head of the LRU list. In order to reduce the overhead, we adopted *flat combining* [?] (to reduce thread lock contention) and ignored the dereferences of an object within 10s (to reduce promotion frequency for extremely hot objects) [20]. However, although an LRU-like policy can reduce the frequency of remote access, it incurs a maintenance overhead of up to 9% due to a huge number of objects.

Of course, the more bits used, the higher accuracy they bring. Atlas allows developers to customize the hotness tracking policy with the two reserved bits in each smart pointer

(Figure 3.2). For our applications, we did not observe significant performance variations between using one and two access bits—likely the ability of distinguishing hot and cold objects is not increased much with two access bits.

3.5 Related Work

Disaggregation. Resource disaggregation has become a trending architecture for datacenters to improve resource utilization. Its key idea is to break the server hardware boundary and unstrand idle resources of remote servers by leveraging advanced network hardware [41, 52]. Existing systems have demonstrated the viability of disaggregated storage [64, 71], accelerators [96, 117, 132], network [110], and memory [46, 109]. For a memory-disaggregated system, memory spans across multiple servers. The efficient data path of Atlas can speed up the data transfer between servers.

Paging-Based Remote Memory. A practical way to deliver remote memory is to leverage the paging system to access remote memory. Google and Meta have reported their successful deployment of such systems in their datacenters [68, 127]. Many optimizations to the kernel data path have been proposed for improved efficiency, including but not limited to bypassing the block layer [15, 101], prefetching more accurately [84], and reducing interference [122]. The design of Atlas is orthogonal to the underlying paging systems and can directly benefit from optimizations within these systems.

Object-Based Remote Memory. Many runtime libraries offer new primitives for object-granularity remote memory management, making them a more efficient alternative for scattered data on far memory. For example, AIFM [103] proposed remote-able data structures, FaRM [37] offered key-value interfaces, and Grappa [90] builds a software distributed memory. Atlas focuses on the cooperative use of its two data paths and benefits directly from existing optimizations.

Emerging hardware. Emerging hardware technologies unlock new opportunities for efficient remote memory. Clio [48], StRoM [111], and RMC [16] offload functionalities to their customized hardware to reduce network traffic. Finally, CXL [3, 44, 72, 73, 134] and Project PBerry [23, 24] enable remote memory access at the cache-line granularity. Atlas directly benefits from the throughput and latency advancements of new hardware technologies. Besides, for hardware solutions with a fixed access granularity, Atlas can improve data locality to improve data transfer efficiency.

3.6 Summary

This chapter presents Atlas, a hybrid dataplane that enables efficient remote memory for bulk data and scattered objects simultaneously. Atlas outperforms both the state-of-the-art object-based and paging-based remote memory systems.

CHAPTER 4

Firework: Efficient and Transparent Compute Scaling with CXL Shared Memory

Multi-threading is key to application scalability, but extending this scalability beyond a single physical node remains difficult—it typically demands significant rewrites of application logic and reliance on specialized systems tailored to particular distributed workloads. This chapter introduces Firework, the first system that can run unmodified, multi-threaded C/C++ applications seamlessly across multiple servers by leveraging hardware-coherent CXL 3.0 shared memory. A central challenge lies in supporting both *transparency* and *efficiency*. For transparency, Firework adopts a co-design between the OS kernel and a runtime layer, allowing kernels across participating servers to coordinate page-level data sharing and migration between local and shared memory, without requiring any application-level changes. For efficiency, Firework minimizes page sharing by judiciously sharing and unsharing pages based on dynamic access patterns. Extensive evaluation across a suite of cloud applications demonstrates promising scalability results.

4.1 Overview

With the rise of cloud computing, the computational model is undergoing a fundamental shift—from single-node, multi-threaded execution to a distributed model that spans multiple servers. This shift allows applications to scale by leveraging the combined resources of many servers. While scaling out (as opposed to scaling up) offers significant advantages—such

as being able to utilize commodity hardware and provide applications with the illusion of unlimited resources—it requires developers to rewrite applications from scratch using custom APIs [76] or distributed frameworks [34, 88, 133], posing a barrier to legacy code adoption. A general solution for seamlessly executing multi-threaded programs across multiple nodes remains an open challenge.

At the heart of this challenge lies the question of how to efficiently share memory across multiple nodes. When multiple threads run on a single server, they synchronize using well-defined locking mechanisms, and memory coherence is ensured by the host CPU. However, once memory is distributed across nodes, coherence is no longer guaranteed, requiring special mechanisms to ensure that loads and stores across servers follow a consistent order—as if they were operating on the same machine. Distributed shared memory (DSM) systems [22, 62, 82, 89, 108, 115, 124] rely on software-based coherence mechanisms to invalidate dirty copies of memory objects. However, most DSM systems require program rewriting and also incur prohibitively high performance overhead. DRust [82], a recent DSM system, mitigates this overhead by exploiting Rusts ownership types. However, its approach is not broadly applicable to programs that are not written with such ownership semantics in mind.

The emergence of Compute Express Link (CXL) [3] has unlocked new opportunities for designing efficient memory systems. While earlier research primarily explored CXL’s capabilities for memory tiering [136] and pooling [74], the introduction of CXL 3.0 [10] brings a significant advancement: hardware-supported cache coherence across the hosts. Specifically, when a memory device is connected to multiple compute nodes via CXL 3.0, writes made by one node into the device become immediately visible to others, with coherence automatically managed by the CXL hardware—eliminating the need for software intervention. This capability enables hosts to share device memory seamlessly and efficiently, offering a new perspective [134] on distributing memory-sharing threads across multiple servers.

This chapter explores how to build a distributed system using coherence-enabled CXL shared memory, allowing legacy multi-threaded programs to run transparently (*i.e.*, without

code changes) and efficiently (*i.e.*, with minimal communication overhead) across multiple servers. This enables applications developed for single-server executions to automatically leverage remote resources upon a load burst.

A straightforward idea is to modify the memory allocator to allocate all objects directly into CXL memory. However, this naïve strategy comes with major drawbacks—CXL memory incurs 20%-3× higher access latency and 3-10× throughput reductions compared to DRAM [3, 74]. Allocating all objects in device memory would thus introduce substantial performance overheads (*e.g.*, more than 10×), undermining the benefits of scaling. An alternative is to provide an annotation library that lets developers specify object placement at each allocation site. Unfortunately, this too falls short: large applications often contain thousands of such sites, making manual annotation impractical. Moreover, objects frequently exhibit dynamic access patterns that vary across execution phases and input data, making static placement decisions inherently suboptimal (see §4.2).

Firework. We built Firework, a runtime-kernel co-design that efficiently and transparently scales out a program on top of CXL 3.0. Given an unmodified single-node C/C++ program, Firework seamlessly executes it in a distributed environment by launching threads across multiple servers and enabling them to communicate through shared memory devices connected via CXL 3.0. Its effectiveness stems from two core capabilities: coordinating compute nodes to share data at the *page granularity* for full transparency, and making intelligent decisions about what data should be shared (and unshared) to optimize performance. These capabilities give rise to three key challenges, which we outline below.

The first challenge is *what to share*. Since programs operate on objects, it may seem natural to place shared objects—those accessed by multiple threads—into the shared memory device. However, managing memory at the object level requires users to manually identify variable-length objects within the application and handle their migration between device and host memory, which is both labor-intensive and error-prone. To allow unmodified single-node

applications to run *as-is*, we shift the focus from user-space objects to kernel-space pages, which are automatically managed by the operating system, independent of the application logic.

Identifying pages for sharing requires a node that accesses a remote page to be able to identify the page’s location. To this end, Firework employs a partitioned address space model, where each node is allocated a unique virtual address range backed by its local physical memory. Each node has visibility into the entire address space, including its own and those of other nodes. Accesses to a node’s own partition are served locally through DRAM, whereas accesses to a partition owned by another node trigger a page fault; Firework handles the fault by initiating the page sharing process on the target node. Unlike the nodes, the shared CXL memory device does *not* have a separate virtual partition; instead, it stores pages migrated from each node’s local memory while preserving their original virtual addresses.

The second challenge lies in *how to share pages*. When a page contains data accessed by multiple nodes, it is migrated from the local DRAM of its current host into the shared CXL memory device. Firework adopts a demand-driven strategy for identifying such pages, leveraging the existing paging mechanism. All pages are initially allocated in the local memory of the allocating threads. As discussed earlier, if node A accesses an address *arr* that currently resides in node B’s memory, a page fault is triggered. Node A then handles the fault by requesting B to migrate the physical page backing *arr* into the shared CXL memory. This process resembles traditional page migration [129], but in a distributed context: both A and B update their respective page table entries (PTEs) for *arr* to point to the new physical location in shared memory, while the virtual address remains unchanged. This coordination relies on a carefully designed distributed synchronization protocol, described in §4.3.3.

This approach requires each node to determine the physical location of a page given a virtual address. In a distributed setting, this is non-trivial—due to migration, the shared memory device can contain pages from multiple nodes, all retaining their original virtual addresses tied to their respective allocating nodes. The situation is further complicated by

page unsharing (discussed shortly), where pages may be moved back from the shared device to a node’s local memory. As a result, virtual addresses alone are insufficient to locate pages physically. To address this, Firework maintains a *page-sharing map* (PSM) within the shared memory device, which functions as an extended page table, allowing each node to locate non-local pages when needed.

The third challenge is *how to minimize the number of shared pages to mitigate performance degradations*. Sharing pages introduces overhead—CXL accesses are slower than local DRAM, especially when the memory device is *remote* (e.g., connected to a remote NUMA node relative to the accessing thread or a rack-scale fabric switch).

Moreover, real-world applications often go through multiple execution phases, each with distinct memory access patterns. As access patterns and data locality change across phases, pages that were shared earlier may become exclusively accessed by a single node, offering opportunities for localization. To illustrate, consider producer-consumer workloads like MapReduce. In the map phase, map threads (*i.e.*, producer) generate intermediate data that is shared across the reducers, while, in the reduce phase, each reducer (*i.e.*, consumer) processes its own partition independently, becoming the sole consumer of the shared pages. In cases where consumer and producer threads are executed on different servers, it is more efficient to keep the data pages local to the producer in the map phase, share them with the consumer once that phase completes, and then localize them to the consumer in the subsequent reduce phase, as the producer no longer accesses them.

To support such workloads, Firework implements *page unsharing*—the inverse of page sharing—which relocates a page from the shared memory device back to a node’s private memory. Firework continuously monitors page access patterns by leveraging the **accessed** bit in each page table entry (PTE), which is automatically set on access. For each physical page in device memory, Firework maintains a *sharing counter* (SC). Periodically, it scans the PTEs of all shared pages and aggregates the **accessed** bits from the PTEs pointing to the same physical page into that page’s SC. Pages with an SC value ≤ 1 are marked as unsharing

candidates as they were accessed by at most one node recently (see §4.3.4).

Results. We implemented Firework in Linux 6.3 and evaluated it using five real-world applications running on real-world CXL hardware. Our results show that Firework scales effectively with the number of servers—approaching the performance of an ideal setup where the application runs on a single server with sufficient compute and memory—and significantly outperforms a baseline that places all allocations in shared memory by an average of 3.8×. Notably, these benefits are achieved without requiring any modifications to the application code.

4.2 Motivation

This section provides extended motivation for Firework’s design by discussing two strawman approaches and showing, empirically, why these approaches fall short.

Strawman #1: Share Everything. A straightforward approach is to allocate all memory in cache-coherent shared memory, allowing every object to be automatically accessible by all nodes. However, this comes with significant performance drawbacks that undermine the benefits of scaling out. First, because of the disparity in latency and bandwidth between local memory and device memory, placing all data in shared memory can severely degrade performance—especially for data accessed exclusively by a single node. Second, CXL 3.0 memory devices have a limited number of cache lines they can track in their directory [11]. Once this limit is reached, accessing a new cache line causes directory entries to be evicted. When the entire working set is indiscriminately shared, the directory fills up quickly, leading to frequent replacements and thrashing.

To quantify this overhead, we ran a Collaborative Filtering application [79] on a system with two NUMA nodes connected via a real CXL memory device (see §4.4 for details). When all objects were placed in device memory, the end-to-end performance degraded by 11.5×

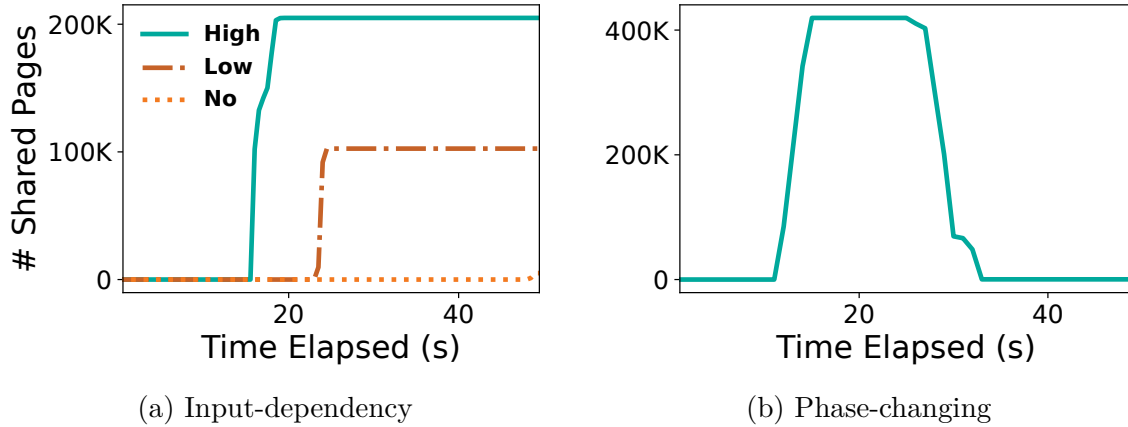


Figure 4.1: Memory sharing patterns vary across inputs and phases: (a) the number of shared pages changes with different skewness levels in the input key space of `DataFrame`—“High” refers to highly skewed input data; (b) the number of shared pages changes as execution phases change in `Collaborative Filtering`.

compared to a baseline that operates entirely on local memory.

Strawman #2: Static Annotation. To improve performance, another straightforward strategy is to differentiate between process-private and shared data, placing only the shared data in device memory. One way to implement this strategy is to design APIs that can explicitly annotate memory allocations creating shared objects [134]. However, this requires heavy code modification, shifting the burden of porting to developers. Furthermore, accurately annotating memory allocations is extremely challenging because an applications memory access patterns are often complex and input-dependent.

We ran a `DataFrame` application [87] where multiple threads process large tables in parallel. These tables are partitioned by key range and each partition is assigned to be processed by one thread, which can lead to workload imbalance if certain key ranges are more frequent than others (*i.e.* skewed distribution). To address this issue, we implemented work stealing, allowing threads that finish early to take tasks from others—triggering data sharing across threads. As a result, the level of data skewness level directly influences the data sharing patterns between threads. As shown in Figure 4.1(a), input skewness impacts both the timing and volume of cross-thread shared data. When the data is highly skewed,

sharing begins earlier and involves more data, whereas with uniformly distributed keys, there is minimal sharing across threads.

Another example is applications with phase-dependent behavior, such as MapReduce, where the same data structure may be treated as globally shared in one phase and thread-private in another. Static annotations at allocation sites are insufficient to capture these dynamic shifts in sharing patterns and cannot adapt memory placement accordingly. As shown in Figure 4.1(b), the amount of shared data varies across different stages of a collaborative filtering MapReduce application [79]. In the earlier stage, threads aggregate partial sums, resulting in extensive data sharing. In contrast, the later stage involves each thread working on a private partition of the aggregated data. As a result, the same data structure exhibits completely opposite sharing patterns within a single execution, driven by the application’s phase transitions.

Key Takeaway. Sharing all objects ensures transparency—requiring no code modifications—but comes at the cost of significant latency overhead. On the other hand, statically annotating object allocations for placement is only practical for programs with simple and predictable behavior. For real-world applications, where memory access patterns vary with inputs and across execution phases, static annotations are insufficient. Effectively scaling out these programs requires novel techniques capable of dynamically adapting sharing strategies in response to observed access patterns.

4.3 Firework Design and Implementation

4.3.1 Execution Model

Figure 4.2 depicts the execution model of Firework: applications run on a cluster of compute nodes connected to one or more CXL 3.0 devices. Each node contains CPUs and local memory, which can only be accessed coherently within that node. In contrast, the CXL

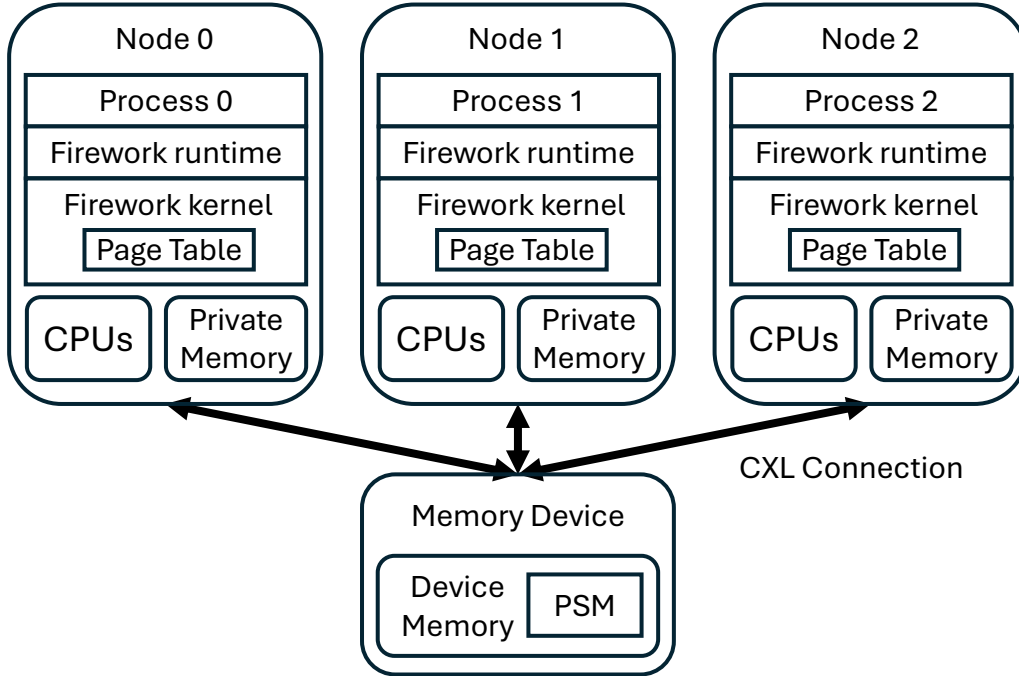


Figure 4.2: The execution model of a Firework application. One process is launched on each node, powered by the Firework distributed runtime and a custom kernel. The page sharing map (PSM) is stored in device memory, acting as an extension of the per-process page table in each node kernel to store shared page metadata and synchronize page sharing operations. device offers globally shared memory with hardware-enforced cache coherence across nodes. Firework allows an unmodified single-node application to run seamlessly in this distributed setup. The application executes as a set of processes within a single shared address space, where each process is mapped to a CPU in the cluster, and each page is physically located somewhere within the cluster.

Each node is assigned a unique node ID and runs an independent OS kernel enhanced with Firework-specific memory management features, such as distributed page migration. Firework includes a distributed runtime responsible for thread scheduling and coordinating page sharing and unsharing across nodes. When an application is launched on a master node, Firework initializes a process on every participating node, with the main process residing on the master. Each process sets up system metadata and launches daemon threads to handle incoming requests from other nodes. Execution begins with the main process on the master

node, and Firework distributes work by issuing remote thread creation requests. Upon receiving such a request, a node spawns a thread to execute the specified function. Memory allocations are initially made in the local memory of the allocating node, but pages may later be migrated to the shared CXL device if sharing is detected—or even to another nodes private memory if the page is subsequently unshared. The physical host of a page—whether a node or the shared device—is referred to as the *provider* of that page. Since each process is bound to a single node, we use the terms process and node interchangeably throughout this chapter.

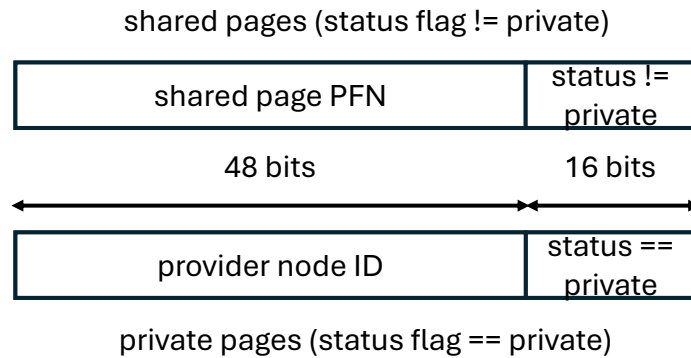


Figure 4.3: Layout of the value fields of a PSM entry.

The shared memory device is primarily used to store shared pages, with a smaller portion reserved for metadata required for synchronization and communication across nodes. Among these metadata structures, the most critical is the *page sharing map* (PSM), which acts as an extended page table tracking the virtual-to-physical mappings of *all shared pages*. This additional mapping structure is necessary because, as discussed in §4.1, each node can access the entire address space—including addresses outside its own partition. In such cases, the page mapping information can not be obtained locally, necessitating a global PSM for resolution and synchronization of page mappings.

The PSM is a key-value store, where each key is a virtual page index and the corresponding value is an entry that describes the pages location. As illustrated in Figure 4.3, each entry takes one of two forms: the primary type of PSM entry stores the location of pages in

shared memory, specifically the page frame number (PFN) of the physical page on the shared device; in special cases, entries may also be created for private pages (after unsharing), in which case the entry records the ID of the provider node instead. These cases are discussed in detail in §4.3.4. Each entry also includes a status flag indicating the pages current state, which is used for synchronization during page sharing and unsharing.

The PSM starts out empty. As execution proceeds and pages are migrated from local memory to shared memory, new entries are added to the PSM with their status set to **shared**. When pages are later unshared and moved back to a nodes private memory, Firework does not remove their entries from the PSM. Instead, it updates their status to **private**, allowing the PSM to continue serving as a reference if those pages are accessed by other nodes in the future. The details of how the PSM is updated during these transitions are covered in §4.3.3 and §4.3.4.

4.3.2 Virtual Address Space

As discussed earlier, each Firework application runs as multiple processes distributed across compute nodes. If these processes perform memory allocations independently, the same virtual address could point to different data on different nodes. This inconsistency would necessitate address translation when passing pointers across nodes, introducing design complexity and additional performance overhead.

To avoid this issue, Firework assigns each process a dedicated partition of the global virtual address space for memory allocation. The memory layout is illustrated in Figure 4.4. All memory allocations by a process are confined to its assigned virtual address range and are backed by that nodes local memory. For instance, in the figure, virtual pages 1-3 were initially allocated from node As local memory. This design ensures there are no address collisions between nodes, allowing addresses to be passed across nodes without translation, while still referencing the same data. Although this design resembles the partitioned global address space (PGAS) model [131], the key difference lies in how pages are bound to partitions.

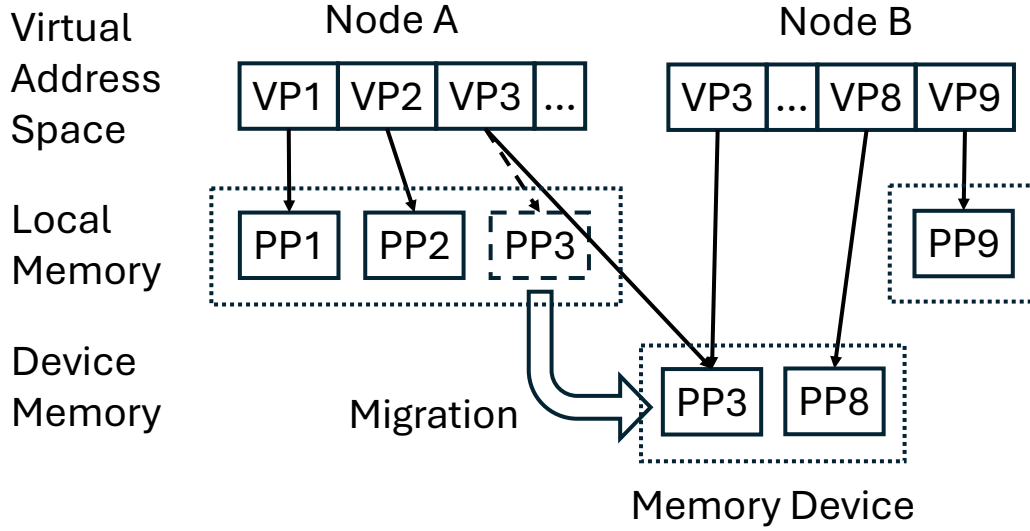


Figure 4.4: Page mapping examples under the global virtual address space. VP/PP denotes virtual/physical page.

In PGAS, the virtual addresses within a partition are consistently backed by the physical memory of the corresponding node throughout execution. In contrast, Firework maintains this binding only at allocation time. As execution progresses, pages may migrate—shared pages can be moved to the device memory and, if later unshared, may even reside in the local memory of a different node.

4.3.3 Page Sharing

On each node, memory allocations are initially conducted using local private memory when a virtual page is first accessed—at this point, the page is only accessible by the node that allocated it (*i.e.*, the provider). When a (thread launched on a) different node (*i.e.*, the accessor) attempts to access that virtual address, Firework initiates the page sharing process. For instance, in Figure 4.4, when node B accesses the virtual address corresponding to Page 3, the page is migrated from node A’s local memory to the shared device memory. Importantly, the virtual address remains unchanged throughout this migration.

The page-sharing mechanism is implemented through a custom page fault handler. When

a fault occurs, Firework’s kernel extension on the accessor first inspects the pages status flag in the PSM in shared memory. This check leads to one of three possible scenarios:

(1) There does not exist an entry in the PSM corresponding to the accessed page. In this case, the page should still be in its provider’s local memory. Given the page has never been shared before, Firework identifies its provider node from the partition that the access hits. Next, Firework creates a new entry for this page in the PSM, indicating that the page is transitioning to the **shared** state. A shared memory PFN is allocated as the target location, and the status is temporarily marked as **share-in-progress** *atomically* to prevent sending redundant migration requests. The accessor then sends a migration request to the provider through an inter-node communication queue (see §4.3.5) and waits for it to complete.

Upon receiving the request, the provider migrates the page from its private memory to the designated location in shared memory, which involves copying the page content and updating the PTE. Once the migration is complete, the status flag is updated to **shared**.

As the provider performs various checks and page migration, the accessor’s page handling thread has to wait. To minimize the overhead of polling or context switching during this wait, we let this thread opportunistically serve pending migration requests from other nodes—the accessor can be the provider of pages accessed by other nodes and hence this thread can serve requests by moving these pages into shared memory. Since a local migration often completes faster than the remote one the handler is waiting on, this does not introduce additional latency to the current page fault.

(2) The status flag is **share-in-progress**. It indicates that the page is currently being migrated, and the accessor must wait for the migration to complete.

(3) The status flag is **shared**, indicating that the page already resides in shared memory and is ready to be mapped.

At the end of the process, the requested page is guaranteed to be in shared memory, allowing the accessor to map it into its own page table. Much like the latency difference

between handling major and minor page faults, the initial non-provider access—and others occurring soon after—incurs higher latency due to the need to wait for page migration. In contrast, subsequent page-sharing faults complete more quickly, as the page can be immediately mapped using the PFN already recorded in the PSM.

Batch Migration. Page migration incurs a non-trivial per-request overhead, largely due to costly operations such as inter-processor interrupts used to flush per-core LRU caches and invalidate TLBs—these can account for up to 90% of the total migration latency in multi-threaded environments. To improve page sharing throughput, Firework supports *batch migration*, allowing multiple consecutive pages to be migrated to shared memory within a single request. The page that triggered the migration is transferred first, while the remaining pages in the batch are migrated asynchronously to prevent blocking the application. This not only amortizes/hides the migration cost, but also *pre-migrates* consecutive pages that will likely be accessed soon as well due to spatial locality. Akin to prefetching [122] performed by the swap system, this batching strategy significantly boosts page sharing throughput, as discussed in detail in §4.4.4.

4.3.4 Page Unsharing

Page unsharing, the inverse of page sharing, moves pages from shared memory back to a node’s private memory. This mechanism is essential for two key reasons: (1) Local memory offers significantly lower latency and higher throughput compared to shared memory. As application phases evolve, some shared pages may become exclusively accessed by a single node, making it more efficient to localize them. (2) The shared memory device has a limited capacity, and hence, a mechanism is needed to reclaim space for newly shared pages. Unlike page sharing—which is triggered on-demand when an unmapped page is accessed—page unsharing is performed in batches periodically.

When the number of available slots in shared memory drops below a predefined threshold,

Firework triggers forced page unsharing—even those that do not meet the usual selection criteria described below—to free up space. This approach is similar to the kernel’s existing swap-out mechanism but differs in how eligible pages are handled: Firework proactively identifies and unshares candidate pages periodically to improve performance, whereas the traditional swap-out mechanism only evicts pages reactively in response to memory pressure.

Candidates Selection. The first step in the page unsharing process is selecting candidate pages. A page may still be mapped in a process’s page table but not accessed recently. We first define *recent accessor* of a page as any process that has accessed it within a *configurable recent time window*, referred to as an *epoch*. Since shared memory is required for pages accessed by multiple nodes, any page with more than one recent accessors is excluded from unsharing to avoid the potential thrashing. Thus, the key criterion for selecting unsharing candidates is identifying pages with at most one active recent accessor. In our implementation, an epoch is defined as the time interval between two consecutive scans of the **accessed** bits.

Our key idea is to periodically scan the **accessed** bit—which is automatically set by hardware on a page access—in each PTE pointing to a shared page within every node’s page table. For each physical page in shared memory, Firework aggregates these bits from all PTEs (across nodes) that reference the same page into a per-page share counter (SC). After the scan is completed on all nodes, each SC reflects the number of distinct nodes that accessed the page during the most recent epoch. Inspired by the second chance algorithm [18], Firework maintains SCs for the two most recent epochs for every shared page. A page becomes a candidate for unsharing only if (1) its SC is ≤ 1 in each epoch and (2) the sum of the two SCs is not zero. If its SC is zero in both epochs, this indicates no recent access, and hence Firework leaves the page in shared memory as long as capacity is not a concern. If the SC is greater than 1 in any epoch, the page has multiple recent accessors, and hence cannot be unshared. If the SC is exactly 1 in both epochs, the accessor node IDs are compared, and the page is considered a valid candidate only if the same node accessed it in both epochs.

The number of recent accessors for each shared page is maintained in an array located in shared memory and updated once per epoch. Firework relies on a dedicated unsharing manager thread in the main process to orchestrate the candidate selection. At the start of each epoch, a daemon thread in each process scans its PTEs that reference pages in shared memory. If a PTEs **accessed** bit is set, it is cleared, and the associated SC for that page is atomically incremented. The node ID of the accessing process is also recorded for future comparison. If the available shared memory goes below a threshold, Firework forces the pages whose SC is 0 to be unshared as well to free memory (by moving them back to their original providers). If unsharing these pages can still not free up enough memory, Firework starts to unshare other shared pages, which could lead to thrashing and significant performance degradation.

The Unsharing Procedure. Once the candidate pages are identified, Firework selects a new provider for each one—*i.e.*, the node to which the page will be migrated. If a page has a single recent accessor, that node becomes the new provider. If the page has no recent accessors but is being unshared due to memory pressure, the node that originally allocated the page is chosen as its new provider.

Unsharing a page involves several transitions of its status flag, as illustrated in Figure 4.5. Before a page can be migrated, it must first be unmapped from the page tables of all non-provider nodes. To initiate this, the unsharing manager sets the status flag of each candidate page to **unmap-in-progress** and sends unmapping requests to all non-providers. These nodes then unmap the page from their page tables. Once unmapping is complete across all non-providers, the manager updates the status flag to **unshare-in-progress** and sends a migration request to the new provider, instructing it to copy the page from shared memory into its local memory.

After the page table scan but before migration, non-provider nodes may still access pages marked as unsharing candidates, creating concurrency issues. In such cases, the page should

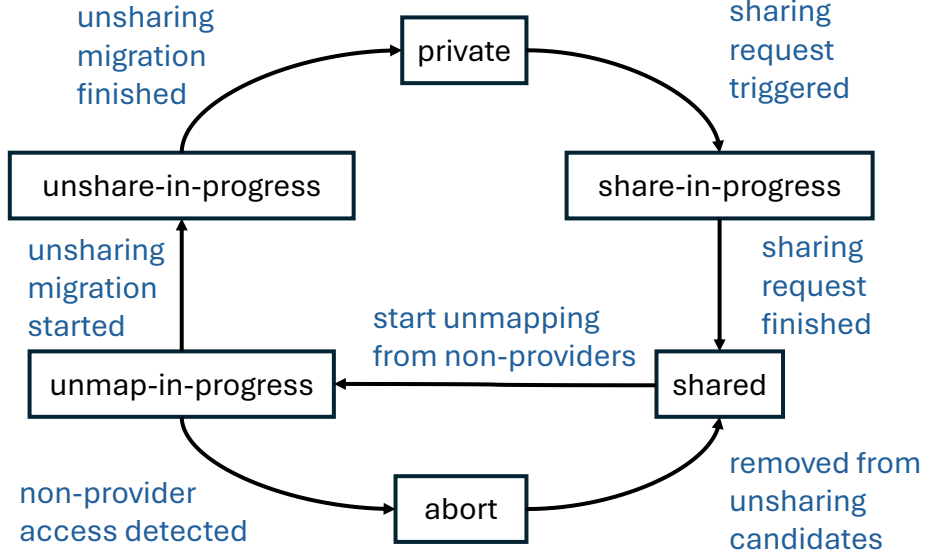


Figure 4.5: State machine of a shared page.

no longer be eligible for unsharing. To handle this, Firework uses several mechanisms to detect these situations and abort the unsharing process accordingly: (1) If the **accessed** bit of a candidate page’s PTE is set at a non-provider during unmapping, the unmapping must abort and update the pages status flag to **abort**. (2) If a page-sharing request is triggered on a non-provider node while the pages status flag is set to **unmap-in-progress**, the node should set the status to **abort** and handle the request as if the page is ready to be mapped. Note that a page-sharing request may still occur even when the page already resides in shared memory. This can happen in two cases: (a) the node is accessing the page for the first time, resulting in a page fault because its page table lacks a PTE for the page; or (b) the node has unmapped the page during the unsharing process but accesses it again immediately after unmapping completes—before the full unsharing procedure finishes.

Before setting the status flag to **unshare-in-progress**, the unsharing manager must first check its current value. If the flag is set to **abort**, the migration request is skipped, and the page is removed from the candidate list, remained in the shared memory. Any node that previously unmapped the page can simply remap it upon accessing it again.

Similar to page sharing, the unmap and migration operations during page unsharing can

also be batched to reduce per-request overhead. A performance analysis of page sharing and unsharing batch sizes can be found in §4.4.4.

4.3.5 Threading, Locking, and Memory Management

Distributed Threading. To support transparent cross-node thread creation, Firework includes a customized version of `libpthread` that enables remote thread spawning. Standard `libpthread` functionalities are extended to work seamlessly with both local and remote threads. For instance, when joining a remote thread, `pthread_join` sends a join request to the corresponding remote node and waits for its completion. Thread-related metadata, including synchronization primitives like spinlocks and barriers, are stored in shared memory to ensure global accessibility. The policy for assigning threads to nodes is decoupled from the underlying mechanism; Firework supports several basic policies such as round-robin and minimum-load scheduling.

Distributed Locking and Communication. Both page sharing and unsharing require a distributed locking mechanism to coordinate between the kernels. The synchronization point for page mapping operations across nodes is a per-entry spinlock mechanism in the PSM, implemented using atomic `compare-and-swap` operations, which are already supported by CXL protocols [134]. To access a PSM entry, a node must first acquire the corresponding spinlock and will busy-poll if the lock is already held by another accessor. Despite relying on spinlocks, PSM queries introduce only minimal performance overhead. This is because each PSM entry is protected by its own spinlock, minimizing contention unless multiple nodes access the same entry simultaneously. Moreover, each query only involves several accesses to the fields within a PSM entry, keeping lock durations short. PSM queries also occur exclusively during already-costly operations, such as page faults and migrations, where they account for only a small fraction of the total processing time (0.3% in our experiments) and thus have negligible impact on overall performance.

Firework uses a set of *communication queues* to transmit requests between nodes. These requests span various operations, including page management, memory reclamation, and thread handling. For each request type, a dedicated queue is allocated on every node, holding incoming requests destined for that node. For certain types of requests—such as page migration or thread creation—the requesting node blocks until the operation is complete, which aligns with the expected semantics of these operations. For instance, a page migration must be completed before the page can be mapped by the requesting node.

Managing Shared Memory. The distributed architecture of Firework introduces complications in memory management. The two major additions Firework provides to the existing memory management libraries are a `remote_free` call that can free memory remotely as well as a page garbage collection (GC) that runs on shared memory.

In a single-node environment, all threads share the same allocator, ensuring that the metadata for each memory region is accessible when `free` is called. In a distributed setting, however, each process maintains its own allocator, which means an object allocated on one node might be freed by a different node that lacks access to the object's metadata. To address this, Firework uses a custom implementation of `free`, which enables objects to be freed by non-allocating nodes. When `free` is called on a non-allocating node, Firework defers the deallocation by writing the request to a *free buffer* in shared memory, owned by the object's allocating node—each node has a distinct free buffer and periodically scans it to process pending free requests in batches.

In a single-node environment, the allocator determines when a physical page can be unmapped and released back to the OS—a mechanism that continues to apply to private memory in Firework applications. For shared pages, Firework introduces a *reclamation buffer* in shared memory, which tracks shared pages marked for reclamation (*e.g.*, during unsharing). A daemon thread in the main process periodically scans this buffer, removes corresponding entries from the PSM, and returns the pages to the shared memory pool.

for future allocation. Since operations like `free` and page reclamation typically fall outside the application’s critical path, a deferred, batched approach is well-suited for both. When memory pressure arises, Firework proactively flushes these buffers to free up space for new allocations.

4.3.6 Remote I/O and Fault Tolerance

Remote I/O. File access support consists of two key components: accessing the file content and managing file descriptors. In data center environments, file content is typically shared via network file systems, making it accessible across nodes in Firework. However, file descriptors encapsulate in-kernel metadata such as file positions and caches, which are not transferable across nodes. As a result, if a file descriptor is passed to a thread on a different node, the receiving kernel lacks the necessary metadata to correctly perform file operations. To avoid this issue, file descriptors should not be shared between threads on different nodes. Instead, each thread should independently open and manage its own file descriptor for any file it needs to access. Future work may explore supporting cross-node file descriptor usage through descriptor migration techniques, as investigated in prior work such as [36].

Firework applications can interact with external applications using network sockets. As with file descriptors, network sockets rely on kernel-level metadata and therefore should not be transferred across nodes. Each thread involved in external network communication should establish its own dedicated connection (*e.g.*, via a shared connection pool). To enhance Firework’s networking capabilities, future work could explore techniques such as IP/socket migration [31] and intelligent request dispatching systems.

Certain stateless system calls, such as `time` and `nanosleep`, function correctly across different nodes. However, applications with strict timing requirements must ensure proper clock synchronization across nodes to maintain accuracy. Our current implementation does not support applications that rely on signal handlers.

Fault Tolerance. Firework provides a simple fault tolerance algorithm that sets global barriers and periodically checkpoints each node’s local memory and thread stacks as well as a snapshot of shared memory, in a separate region of the CXL device at each barrier. Upon a node failure, Firework launches a new node, makes each node read the heap and stack information from the latest checkpoint, and resumes the execution from there. Future work can further improve this algorithm, *e.g.*, by leveraging reference counting and garbage collection techniques proposed in [134].

4.3.7 Dynamic Scaling

Firework supports dynamic membership, allowing nodes to join or leave an application during execution. At initialization, Firework reserves a virtual address range and associated metadata (such as communication queues) to accommodate the maximum number of possible nodes. When a new process joins an already running application, it broadcasts a message to all existing nodes to update the total process count and assign itself an index. Once registered, the new process becomes eligible to receive thread spawning requests. Similarly, a process can leave the application by notifying all other nodes of its intent to go offline. After this notification, other processes stop sending new requests to the departing node. The process then completes any outstanding requests and migrates its private memory pages to the shared memory. Once these steps are completed, the process can safely terminate. Note that any subsequent memory deallocation requests targeting the virtual memory range owned by the offline process will no longer be handled. However, this does not affect application correctness.

4.3.8 Limitations

As previously discussed, Firework is designed to focus on compute scaling and distributed memory management. Its current implementation does not support programs that rely on signal handling, or share file descriptors or socket objects between threads. Since Firework

aims to support general-purpose C/C++ applications—most of which rely on `pthread`s—it does not support thread migration, as each `pthread` maintains kernel-level state that cannot be easily transferred across nodes. However, Firework does support dynamic scaling by allowing new processes to be launched on different servers, as described in §4.3.7. Additionally, address space layout randomization (ASLR) is currently disabled for running Firework applications. However, it can be enabled if the same memory layout is preserved across all nodes [82, 105].

4.4 Evaluation

We implemented Firework by adding/modifying 11150 lines of code for the paging system of the Linux kernel v6.3, the memory allocator and `libpthread` in `glibc` 2.35, as well as a runtime for initialization and cross-process communication. To launch an application on Firework, the Firework runtime and the custom `glibc` need to be linked to the application, and `firework_init` needs to be invoked to initialize the runtime, but no further changes are needed.

4.4.1 Methodology

Setup. Firework was evaluated on a server equipped with two Intel Xeon Platinum 8457C CPUs, configured as two NUMA nodes, each with 512GB of DRAM and connected to an Astera Labs A1000 CXL 2.0 memory expansion card. Since CXL 3.0-capable CPUs are not yet commercially available, running Firework over multiple servers would require a software-based cache coherence implementation, similar to how a DSM system explicitly invalidates cached data via message passing. This can introduce a 5-20× performance overhead [82], much higher than that of the CXL 3.0 hardware-coherent device memory, which is estimated to be ~1.6× [33]. As such, we used multiple NUMA nodes to emulate the distributed setting where the NUMA CPUs provide cache coherence. Previous work [134] used the same

emulation for CXL 3.0 while their experiments were run on a CXL 2.0 platform [14].

Specifically, Firework scales out a multi-threaded program by launching multiple processes on the two NUMA nodes. When these processes access the same physical page in the CXL device, the CPUs recognize the shared access through the physical address and ensure coherence by invalidating stale cache lines on writes. The cost of NUMA-based cache coherence (50%-2×) is estimated to be much closer to the coherence cost of CXL 3.0, compared to a software-based coherence implementation. Note that Firework’s implementation does *not* have any limitations that would prevent it from running on multiple servers.

Apart from using a CXL 2.0 device and NUMA to emulate the hardware-coherent CXL 3.0 memory, all other components of our setup faithfully replicated a true multi-node environment with CXL 3.0 memory devices. All inter-process/node communication was conducted entirely through shared CXL device memory, and no synchronization primitives restricted to a single node (*e.g.*, **futexes**) were used. Since communication does not go through the network stack, the communication overhead in our emulation closely approximates what would be expected in a true multi-server deployment. Note that multiple CPU vendors [12, 33] have confirmed that upcoming hardware-coherent CXL 3.0 devices will offer significantly lower access latency compared to current CXL 2.0 devices. Therefore, with real CXL 3.0 hardware, we expect Firework’s performance to surpass the results presented in this section.

To emulate a realistic environment with multiple CXL devices exhibiting varying performance based on CPU proximity, we let the CPU from each NUMA node access CXL memory attached to both nodes. This allows us to evaluate the performance in a real-world setting where multiple memory topologies co-exist: a node’s local DRAM, CXL memory attached directly to the node (*i.e.*, near memory), and CXL memory attached to a remote node (*i.e.*, far memory), as summarized in Table 4.1. Under Firework, each process migrates pages to its CXL near memory (for efficiency) but accesses pages in both near and far memory. The PSM is allocated in the near memory of the node that runs the main process. This reflects a realistic deployment where each node has accesses to devices at different distances.

Memory Backend	Local Mem	Near CXL	Far CXL
Latency (ns)	120.9	237.4	497.8
1-thread Seq R/W (GB/s)	11.3/8.6	5.0/4.0	3.6/2.3
1-thread Rand R/W (GB/s)	0.32/0.32	0.26/0.23	0.17/0.09
Peak Sequential R/W (GB/s)	230.0/110.4	46.5/25.1	17.5/9.3
Peak Random R/W (GB/s)	21.8/9.9	6.1/3.2	2.3/1.1

Table 4.1: Performance of different memory topologies. R/W stands for read/write.

Application	Dataset	Memory	Characteristics
DataFrame (DF) [87]	h2oai [51]	65.6GB	Input-dependent, Mixed Accesses
Collab Filter (CF) [114]	MovieLens [54]	34.6GB	Phase-changing, Mixed Accesses
PageRank (PR) [99]	Twitter [66]	16.9GB	Phase-changing, Mixed Accesses
Feed Gen (FG) [21]	Facebook [66]	24.6GB	Phase-changing, Random Accesses
Sci Kernel (SK) [107]	Synthetic	5.4GB	Compute Intensive, Stream and Strided Accesses

Table 4.2: Applications used for Firework evaluation.

Applications. Firework was evaluated using a diverse set of five applications spanning a wide range of cloud use cases, including data analytics, graph processing, scientific computing, recommendation systems, and web services. Table 4.2 reports their datasets, working set sizes, as well as characteristics. These applications were developed originally to run as a single-machine multi-threaded process. Our goal is to demonstrate that Firework can efficiently scale them to a multi-process environment without requiring application modifications.

DataFrame (DF) [87] organizes data into rows and columns, much like a relational database table. It supports a variety of analytical operations, such as join, groupby, and slice. To process large DataFrame tables efficiently, data is divided into smaller chunks, allowing each thread to work on a separate portion in parallel.

The application performs a hash join on two input tables generated by h2oai [51]. These tables are divided into chunks, with each chunk containing entries that fall within a fixed

key range. The key space is intentionally skewed so that 25% of all entries are concentrated in a single chunk, referred to as the “big chunk”. At the start of execution, each thread is assigned one chunk from each input table to process. It begins by inserting the entries from one table’s chunk into an intermediate hash table indexed by key. Then, entries from the corresponding chunk of the second table are used to probe this hash table, and all matching key-value pairs are concatenated and written into a result chunk.

Because of the skewed key distribution, most threads complete their assigned chunks quickly and then engage in work stealing to assist with processing the big chunk. In this phase, the intermediate hash table becomes shared among multiple threads. During evaluation, the size of the input tables is scaled with the number of threads, ensuring that each thread processes approximately 400MB of data per input table. Throughput is calculated by dividing the total input size by the per-core CPU time consumed.

Collaborative Filtering (CF) [114] is a widely used technique in recommendation systems, where the goal is to derive latent features for both users and items to compute similarities and generate recommendations. This feature extraction is typically implemented using a matrix factorization algorithm, often parallelized as a multi-stage MapReduce task[79]. Across stages, the data is re-partitioned, resulting in different memory access patterns.

The input is a $128K \times 128K$ sparse matrix representing user-item interactions, with one billion non-zero entries. The objective is to decompose this matrix into two lower-rank matrices: user and item factors of size $128K \times 4K$. Each thread begins by loading the sparse matrix and computing partial sums of initial factor values, which are then aggregated and regularized. The factors are subsequently refined through multiple optimization iterations. In each iteration, threads compute gradients for a subset of factor entries, update them accordingly, and then perform global synchronization to ensure consistency.

PageRank (PR) [99] is a foundational graph processing algorithm used to rank web pages in search engines. It models the web as a graph, where vertices represent web pages and edges represent hyperlinks. Each vertex starts with an initial weight, and through iterative

updates based on the link structure, the algorithm refines these weights to compute the final rankings. In the multi-threaded version, each thread is responsible for processing a subset of vertices or edges, while all threads share a global rank array that stores the weights for every vertex. The application begins by loading a 11GB input graph and then assigns an initial rank to each vertex in a global rank array. Then, the ranks are updated in iterations. For each iteration, each thread computes intermediate rank values while accessing a global rank array for a non-overlapping range of vertices, as assigned by the main process. Finally, the global rank array is updated by aggregating the intermediate rank values, marking the end of an iteration and updating the rank values for all vertices. The global rank array is the main data structure shared across all threads.

Feed Generation (FG) [21] generates time feeds using social graph data. To serve each request, the system first fetches metadata from an in-memory key-value store with 128M entries, then retrieves the corresponding 8KB graph from an in-memory store, and finally performs graph traversal on it to identify posts to include in the feed. Multiple threads handle requests in parallel while accessing the globally shared key-value store, while the graph partition is private to each thread.

Scientific Kernel (SK) [107] implements a collection of core linear algebra operations, including matrix multiplication and eigenvalue computation, which are foundational to many scientific computing applications. It operates on $8K \times 8K$ input matrices. Each task is divided among multiple threads, with each thread responsible for processing a portion of the workload. At the start of the execution, each thread initializes its thread-local buffer for its slices of input matrices and intermediate results, then performs its computation. In the end, each thread writes its share of results into a global output region with synchronization.

Baselines. The performance of Firework was evaluated against two baseline systems (§4.4.2). The first, referred to as *share-everything*, allocates all memory directly in shared memory and maintains a global virtual address space with a fixed virtual-to-physical page mapping.

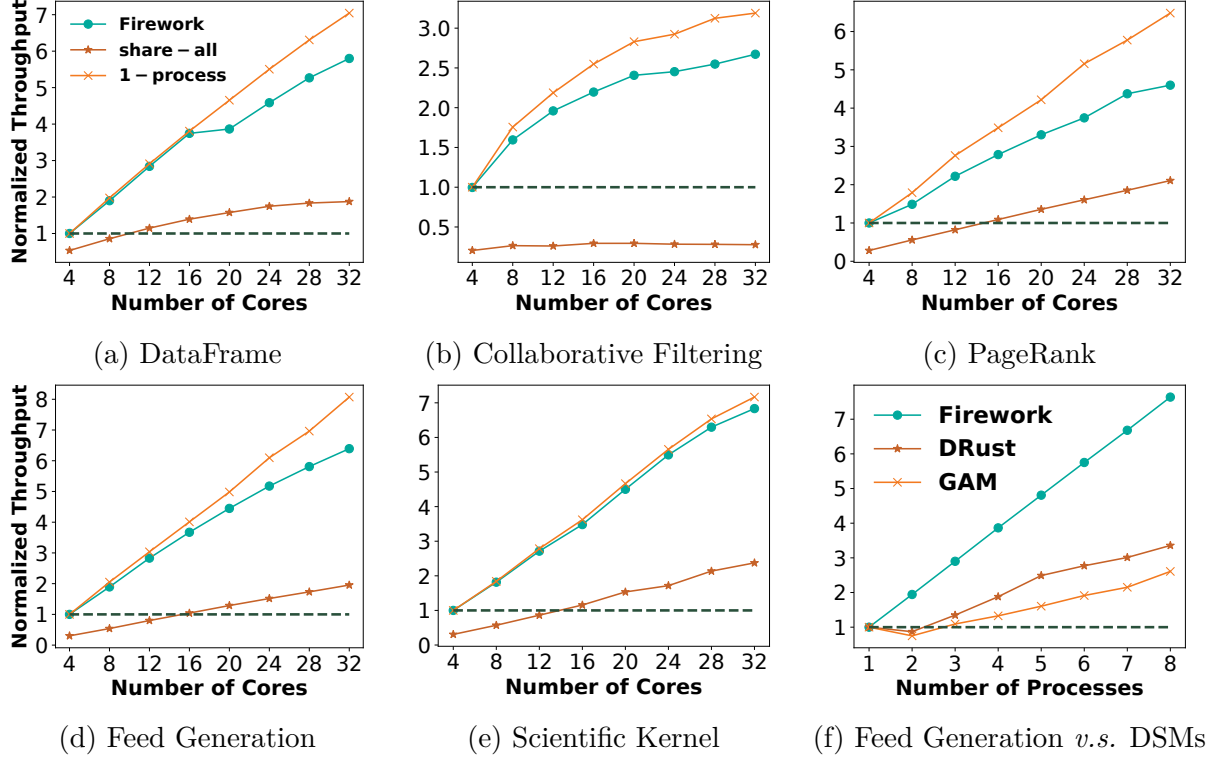


Figure 4.6: (a)-(e) Throughput and scalability comparison among Firework, share-everything, and single-process, normalized to the throughput of a single process with 4 cores (*i.e.*, the dark green dash line). (f) scalability comparisons between Firework, DRust [82], and GAM [22] on Feed Generation

This allows the entire working set to be coherently shared across processes and serves as a strawman solution for utilizing CXL shared memory without optimizations. The second baseline, *single-process*, uses the same total amount of compute but runs as a single process, avoiding shared memory and inter-process communication overhead entirely. The second configuration serves as an idealized reference point to quantify the overhead introduced by Firework.

Since existing systems that scale out single-machine programs typically require either rewriting applications using specialized APIs [22, 134] or implementing them in specific languages like safe Rust [82], there is no directly comparable system that can run unmodified C/C++ programs *as-is*. To enable a meaningful comparison, we selected one application, Feed Generation, and ported it into two alternative versions: one using the API of GAM [22],

a state-of-the-art DSM system for C/C++ programs, and another rewritten in Rust to run on DRust [82], a high-performance DSM system for unmodified Rust applications. We used this setup to compare Firework against both GAM and DRust in §4.4.3. Due to the time-intensive nature of porting, this experiment was limited to a single application.

4.4.2 Throughput and Scalability

We begin by comparing the end-to-end throughput and scalability of Firework against the share-everything and single-process baselines. Our experiments were conducted on three systems, each with an increasing number of CPU cores. Since both Firework and share-everything scale by launching processes, we let each process use a fixed number (4) of cores, and the number of processes scales with the total core count. In contrast, the single-process setup scales by launching threads, and hence its core count increases as the number of threads increases. The results are presented in Figure 4.6. Overall, Firework consistently outperforms share-everything across all applications, achieving an average speedup of 3.8×, thanks to its ability to dynamically share and unshare pages. This allows Firework to place each page in the most appropriate memory backend, thereby minimizing access latency. Moreover, Firework’s performance closely approaches that of the ideal single-process setup, incurring an average overhead of 21%—primarily due to the performance gap between device and local memory.

DataFrame (DF). As shown in Figure 4.6(a), Firework matches the scaling performance of the single-process baseline up to 16 cores (4 processes), since work stealing remains confined within each process and no cross-process page sharing occurs. Once the system scales to 5 or more processes, the size of the big chunk exceeds what threads within a single process can handle. As a result, work stealing extends across processes, triggering page sharing on the intermediate hash table, with 820MB of data migrated to shared memory. Despite this, Firework delivers a 5.8× throughput improvement when scaling from 1 to 8 processes, with only a 17% slowdown compared to the ideal single-process baseline. Additionally, Firework

outperforms the share-everything baseline by up to 3.1 $\times$. In the share-everything setup, all table accesses incur device memory overhead. In contrast, Firework only migrates the intermediate table associated with the big chunk to device memory during cross-process work stealing. All other table accesses are handled through DRAM, minimizing overhead.

Collaborative Filtering (CF). Due to its high memory intensity and frequent global synchronization, CF is the least scalable application in our benchmark suite. Nevertheless, Firework achieves a 2.7 $\times$ speedup when scaling from 1 to 8 processes, with a 19% performance overhead compared to the single-process baseline. During execution, approximately 1.6GB of data is actively shared and unshared, primarily consisting of factor matrices. These factors are shared during the aggregation and regularization phases, but are later partitioned for optimization, where each thread operates exclusively on its private partition. This creates an opportunity to improve performance by migrating those partitions back to private memory. Page unsharing yields an additional performance gain of up to 1.8 $\times$, as detailed in §4.4.4. In contrast, the share-everything approach suffers from severe scalability issues due to the bandwidth constraints of device memory, resulting in a 9.6 $\times$ slowdown.

PageRank (PR). When scaling from 1 to 8 processes, PR achieves a 4.6 $\times$ increase in throughput, with a 2040% overhead relative to the single-process baseline. While only 318MB of memory—primarily the global rank array—is shared, this array is subject to frequent and random access by all threads, which amplifies the latency impact of slower device memory. Despite this, Firework still delivers up to a 2.7 $\times$ speedup compared to the share-everything baseline.

Feed Generation (FG). All threads query the key-value store using a uniformly distributed key space, requiring the entire key-value store to be migrated to shared memory. However, compared to the key-value store, the size of graph data is much larger. Firework is able to detect that the graph data does not need to be shared (as each partition of the graph is accessed exclusively by a node), allowing it to retain that data in private memory and achieve significantly better performance during graph traversal, which is both highly

random and memory-intensive. Overall, Firework delivers a $6.4\times$ speedup when scaling to 8 processes, outperforms share-everything by $3.5\times$, and incurs only a 726% overhead compared to single-process.

Scientific Kernel (SK). Firework intelligently identifies and migrates only the shared data—namely, the global input and output matrices—while allowing thread-local buffers to remain in local memory for most of the accesses, incurring no performance penalty. As a result, Firework achieves a $6.8\times$ speedup when scaling from 1 to 8 processes, with only a 5% overhead compared to the single-process baseline. In contrast, the share-everything approach places both shared and private data in device memory, making the entire execution subject to device memory overhead and resulting in $2.9\times$ lower throughput than Firework.

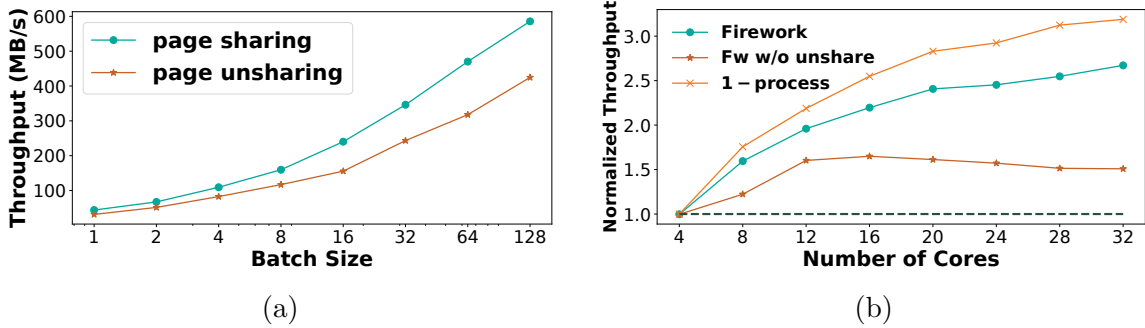


Figure 4.7: (a) single-threaded page sharing and unsharing throughput under different batch sizes; (b) CF performance comparison between Firework, Firework without unsharing, and single-process.

4.4.3 Comparing Firework with DSMs

This subsection presents a comparison between Firework and two DSM systems that implement software-based cache coherence: GAM [22] and DRust [82]. Since GAM and DRust were evaluated on a cluster using RDMA for inter-node communication—unlike Firework, which uses shared device memory—directly comparing their raw throughput is not meaningful; in fact, Firework’s raw throughput is significantly higher than that of both. To address this, we instead compare their normalized scalability, measuring how well each system improves as more compute resources are added. Specifically, we normalize the throughput of

each system relative to its own single-process baseline.

As shown in Figure 4.6(f), Firework demonstrates significantly better scalability than the DSM systems, achieving an up to 1.6× improvement over DRust and an up to 1.7× improvement over GAM. To understand the root causes of these performance differences, we analyze FG’s execution in two parts: private graph traversal and KV store querying.

In the graph traversal phase, all three systems showed comparable performance, as this stage primarily involves local computation without cross-node synchronization. Both GAM and DRust cache data objects locally, and most memory accesses are satisfied from each nodes local DRAM.

The primary performance gap emerges in the KV store querying phase, where the KV store is shared across nodes. GAM enforces cache coherence by explicitly sending invalidation messages to each node, incurring substantial overhead. Although DRust avoids explicit invalidations by relying on Rust’s ownership model, it still needs to transfer key-value pairs across nodes at the object level, which adds communication cost. In contrast, Firework moves the KV store into shared device memory, incurring a limited number of page migrations. This approach enables efficient access throughout execution while minimizing communication overhead.

4.4.4 Performance Drilldown

This subsection examines the costs and benefits of several key design decisions. Due to space limitations, the experiments focus on CF; however, the insights and trends observed are representative of all evaluated applications.

Dynamic Sharing Patterns. Figure 4.1(b) illustrates the volume of cross-process data sharing throughout the execution of the CF application. As previously discussed in §4.2, the factor data exhibit varying sharing patterns across different phases, leading to both page sharing and unsharing events. Page unsharing is performed once per epoch. We found that

the majority of pages are selected as candidates and unshared in the second epoch after the start of the new phase, as these pages have not been accessed by more than one process in the last two epochs. A smaller portion is selected in the subsequent epoch, which accounts for the two noticeable drops in shared data shown in Figure 4.1(b).

Sharing/Unsharing Performance. Page sharing and unsharing are the primary sources of overhead in Firework, aside from the inherent latency and throughput overhead incurred by device memory. As described in §4.3, their throughput is highly sensitive to the batch size—that is, the number of pages migrated or unmapped in a single batch. To demonstrate the benefits of batching, Figure 4.7(a) shows the end-to-end throughput of single-threaded page sharing and unsharing with varying batch sizes. As the batch size increases from 1 to 32, the throughput improves by 7.9× for page sharing and 7.8× for page unsharing. Notably, page unsharing achieves about 29% lower throughput than page sharing at the same batch size, primarily due to additional unmap operations and more complex synchronization requirements.

Selecting the appropriate batch size for page sharing and unsharing involves different considerations. In the case of page sharing, batching is performed over pages with contiguous virtual addresses. However, not all pages in a batch may actually be accessed by a remote node, so a large batch size can lead to unnecessary sharing (*i.e.*, false positives). Therefore, the batch size for page sharing should remain relatively small to limit this overhead. In contrast, page unsharing is guided by profiling data, and the set of pages to be unshared in each epoch is determined collectively. Since these pages are known to be good candidates, larger batch sizes can be used without increasing the risk of false positives. In our implementation, we used a batch size of 32 for page sharing and 128 for page unsharing.

Effectiveness of Page Unsharing. Page unsharing is a crucial optimization in Firework that enhances memory performance and, in turn, improves overall throughput. As illustrated in Figure 4.7(b), enabling page unsharing consistently boosts performance across all config-

urations, yielding a 1.2× to 1.8× speedup. When running with three or fewer processes, the primary benefit comes from lower-latency local memory accesses after unsharing. As the number of processes increases beyond three, the device memory’s limited random access throughput becomes a performance bottleneck. Without unsharing, Firework is unable to scale effectively under these conditions, while placing factor data in local memory avoids this constraint and continues to deliver scalable performance.

Page unsharing introduces an overhead of 5 – 9% in execution time. Nevertheless, this cost can be easily offset by the substantial performance gains, resulting in improved overall throughput.

4.5 Related Work

CXL-Based Memory Tiering, Pooling, and Sharing. Recent research on CXL-based memory tiering explores how to effectively manage heterogeneous memory systems composed of traditional DRAM and CXL-attached memory devices, which typically have higher latency and lower bandwidth. Memstrata [136] uses a hybrid approach that combines hardware-managed tiering with software-managed performance isolation to reduce overhead in managing tiered memory in virtualized environments. Lee *et al.* [70] developed a SDK for managing CXL 2.0-enabled tiered memory. Nomad [129] proposed *nonexclusive memory tiering*, a page management strategy that retains a copy of pages recently promoted from slow memory to fast memory to mitigate memory thrashing. Pond [74] is a CXL-based memory pooling system that addresses the challenge of underutilized memory—often referred to as “stranded memory”—which arises when virtual machines do not fully utilize their allocated DRAM. CXL-ANNS [60] is a system that uses CXL-based memory pooling to perform efficient approximate nearest neighbor search at scale. CENT [47] uses CXL to extend memory capabilities to support LLM inference. Zhang *et al.* [134] built a failure-resilient memory management system for CXL 3.0-based shared memory.

Disaggregated and Remote Memory. Memory disaggregation and remote memory techniques are a promising approach to scaling applications out of a single machine. Their key idea is to connect a host server with large memory pools [42, 56, 63] via fast datacenter network, which can be accessed by applications via OS kernel [15, 24, 101, 109, 122] or software runtimes [30, 49, 80, 104, 120, 121, 123]. However, they do not provide cache coherence and hence cannot support compute offloading.

Compute Scaling. Researchers have explored various programming languages and abstractions to simplify distributed programming. Munin [19] introduced a type system that distinguishes between local and global pointers and uses type checking to track pointer sharing. X10 [29, 53] and UPC [38] provide function offloading interfaces and type annotations to reduce runtime overhead in distributed environments. More recent systems like Ray [88] and Nu [105] propose new abstractions for distributed execution. However, unlike Firework, these systems require significant effort to adapt existing applications, as they discourage fine-grained memory sharing. Systems like Clio [48], StRoM [112], and RMC [16] reduce remote memory access latency by offloading operations to custom hardware. Meanwhile, platforms such as Concordia [124] provide hardware-level support for memory coherence.

Distributed Shared Memory. Distributed cache coherence protocols and their implementations for DSM have been widely studied since the 1980s [25–27, 39, 50, 75, 77, 86, 91–93, 113]. Systems like Munin [19] and TreadMarks [17] introduced relaxed consistency models and simplified coherence protocols to reduce the overhead associated with maintaining coherence. More recent DSM systems have taken advantage of modern hardware technologies such as RDMA [22, 62, 89, 108, 115, 134], or language features like ownership types [82] to enhance performance and scalability.

4.6 Summary

Firework enables unmodified C/C++ applications to run efficiently across multiple servers. By leveraging CXL 3.0 hardware for transparent coherence and combining it with a lightweight, kernel-assisted runtime for dynamic page management, Firework eliminates the need for complex application rewrites. Its design offers a practical and high-performance path forward for scaling legacy and modern multi-threaded applications beyond a single node.

CHAPTER 5

Conclusion

5.1 Key Contributions

Memory disaggregation has been reckoned as a promising direction to improve data center memory efficiency, which introduces additional tiers of memory other than host DRAM in monolithic servers. These additional memory tiers have different capability and performance characteristics, introducing new challenges in memory management in aspects such as data placement and data transfer across different tiers. Therefore, comprehensive memory management systems for memory-disaggregated data centers have been proposed. However, data center applications exhibit diverse and dynamic memory access behaviors, but each of these existing solutions makes unrealistic assumptions about the memory access behaviors and tunes its performance based on such assumptions. As a result, none of these systems is able to deliver satisfactory performance consistently with diverse data center workloads.

Ideally, the memory manager for memory-disaggregated data centers should be able to identify diverse and dynamic memory access behaviors, and make adaptive decisions on how data placement and transfer should be conducted, so it can always produce the best performance regardless of the diversity and dynamicity of memory access behaviors. Based on this insight, this dissertation introduces two systems, each working on a different memory disaggregation setting, where it makes adaptive decisions based on online monitoring of memory access behaviors and transfers data across different memory tiers accordingly.

Chapter 3 targets an RDMA-based remote memory setting. It identifies the diverse and

dynamic memory access localities, and urges that the data transfer granularity should be variable based on the locality of remote data, so the throughput is maximized for bulk data, and the amplification is minimized for fine-grained data. A comprehensive evaluation of various cloud applications proves that a hybrid data transfer path greatly improves the system performance, outperforming state-of-the-art systems with fixed data transfer granularities.

Chapter 4 aims to boost cross-node multi-threaded applications on CXL shared memory. It detects data sharing across nodes and shares data at the page granularity on demand. It also predicts potential changes in the data-sharing behaviors in the future via light profiling, and improves memory performance by migrating data back to faster host memory. By capturing the dynamic sharing patterns to make wise data placement decisions, it substantially outperforms static data placement, while maintaining transparency to the application programmer.

Collectively, these systems showcase the imperative of adaptive memory management, especially for memory-disaggregated data centers, where memory becomes hierarchical and memory capabilities become more diverse. This dissertation not only addresses practical challenges for disaggregated memory efficiency, but also lays the groundwork to apply the philosophy of adaptiveness to the bigger picture of data center memory systems spanning beyond memory disaggregation.

5.2 Future Directions

Modern data center memory systems are undergoing a significant transformation to meet the demands of increasingly diverse and data-intensive workloads. This evolution is characterized by a shift towards hierarchical and heterogeneous memory architectures.

On the one hand, contemporary data centers are moving beyond traditional homogeneous memory setups, embracing a layered approach that integrates various memory technologies. This includes combining high-speed DRAM, high-bandwidth memory (HBM), disaggregated

memory, persistent memory technologies like Intel Optane, as well as non-volatile memory (NVM) including SSDs and traditional hard drives. Such heterogeneity allows for a balance between performance, capacity, and cost, catering to a wide range of application requirements.

On the other hand, designing memory management systems now involves navigating a complex landscape of trade-offs. The most classic one is the trade-off between capacity and performance. Memory management systems need to be able to identify the characteristics of each data structure, and then make wise data placement and transfer decisions. Moreover, different memory technologies provide diverse capabilities beyond capacity and performance, dramatically increasing the number of metrics to consider when designing memory management systems. For example, with CXL cross-node shared memory, data concurrently accessed by multiple servers must be kept in the shared memory to ensure global accessibility. When compute is available for a specific memory backend, the system needs to figure out what data should be stored on the backend to enable compute offloading with better parallelism and reduced memory access latency.

Existing systems were not originally designed to accommodate the complexities of modern hierarchical and heterogeneous memory architectures and act adaptively. This mismatch can lead to suboptimal utilization of memory resources, increased latency, and challenges in maintaining data consistency and coherency across different memory tiers. Adapting legacy systems to the new memory paradigms requires significant re-engineering efforts, and we believe there are multiple promising directions:

Designing Around Hardware Intelligence. Modern memory technologies provide novel capabilities, including but not limited to coherent sharing, near-memory compute, and persistency. Memory management systems should be aware of such features and exploit adaptiveness to enable new opportunities for performance improvements. For instance, CXL memory enables globally coherent/shared regions across nodes, but bulk data may be better managed

via software-controlled coherence to avoid fine-grained back-invalidation overhead. An adaptive system can dynamically switch between hardware-coherent and software-managed regions based on access patterns. Another example is near-memory compute, such as compute-capable CXL devices, SmartNICs, and compute cores on servers providing RDMA remote memory. It enables offloading operations to reduce latency and traffic. An adaptive controller can auto-detect when a piece of data is better processed on near-memory compute and offload them to the memory backend [30, 80, 103, 120]. When designing such systems, the hardware limitations also need to be kept in mind, such as the coherent memory capacity of CXL memory devices and the write amplification behaviors of non-volatile memory. Failure to optimize the system based on these limitations can lead to substantial performance overhead. In summary, such adaptivity can yield huge benefits by tuning memory placement, coherence enforcement, and compute offloading at runtime—without requiring applications to be manually engineered for every backend.

Consolidating Across Different Memory Layers for Synergy. Disaggregated data center memory is a rich tapestry—no single tier satisfies all performance, capacity, and access patterns. Yet, when systems consolidate adjacent hierarchy layers, they unlock powerful synergies that improve scalability and performance. For example, Rcmp [125] demonstrates a hybrid design combining CXL-based memory pools (within racks) and RDMA-based accesses (across racks). It dynamically routes hot-page accesses via low-latency CXL, while leveraging RDMA for capacity beyond rack boundaries. Another attempt is the unified memory architecture exemplified by Nvidia GH200 [13]. It integrates CPU host DRAM with GPU device HBM over cache-coherent high-speed fabrics, presenting a unified address space across CPU and GPU, which enables transparent zero-copy sharing, dynamic over-subscription, and flexible data placement between the two memory layers. By adaptively exploiting these consolidated layers, memory managers make wise decisions on memory placement and data sharing, which enhances the system scalability while preserving performance guarantees.

Combining Automatic and Workload-Specific Semantics. Automatic memory management relieves programmers by collecting runtime metrics and making decisions online. However, some workload semantics—e.g. this dataset is solely owned by this thread or this object is read-only after initialization—are hard or expensive to detect automatically. At the same time, many real-world workloads are domain-specific, offering fertile ground for programmer-provided annotations that complement the runtime system. For example, DRusts affinity pointers batch data to be transferred together to reduce network round trips and improve data transfer throughput. [82]; Firework can be extended to include hints when spawning threads to co-locate new threads and data the threads will access, in order to reduce cross-node data sharing; When working with machine learning workloads, ML practitioners can also annotate tensors to prompt intelligent spilling under memory pressure to ensure critical data is not spilled by mistake with the default spilling policy. By combining automatic profiling (e.g. hotness, access patterns) with lightweight programmer hints, an adaptive manager can converge on near-optimal data placement, while still leveraging developer knowledge for significant performance gains.

These future directions illustrate a vision for modern data centers where adaptivity is central. Together, they outline a roadmap toward memory systems that self-optimize based on workload structure, hardware affordances, and dynamic access behaviors—driving both performance and resource efficiency in a heterogeneous and disaggregated memory environment.

Bibliography

- [1] Cxl and gen-z iron out a coherent interconnect strategy.
<https://www.nextplatform.com/2020/04/03/cxl-and-gen-z-iron-out-a-coherent-interconnect-strategy/>.
- [2] Meta CloudLib. <https://cachelib.org>.
- [3] Compute express link. <https://computeexpresslink.org>.
- [4] Nyc taxi trips - exploratory data analysis. <https://www.kaggle.com/code/kartikkannapur/nyc-taxi-trips-exploratory-data-analysis/notebook>.
- [5] Facebook Folly RCU Library. <https://github.com/facebook/folly/blob/main/folly/synchronization/Rcu.h>.
- [6] std::weak_ptr. https://en.cppreference.com/w/cpp/memory/weak_ptr.
- [7] Memcached - a distributed memory object caching system. <http://memcached.org>, 2020.
- [8] Konect networks data. <http://konect.cc/networks/>, 2021.
- [9] free c++ class library of cryptographic schemes. <https://www.cryptopp.com/>, 2022.
- [10] Compute express link 3.0. https://computeexpresslink.org/wp-content/uploads/2024/01/CXL_3.0-Specification-Release_FINAL-1.pdf, 2022.
- [11] Compute express link 3.0 specification. <https://computeexpresslink.org/wp-content/uploads/2024/02/CXL-3.0-Specification.pdf>, 2022.
- [12] Compute express link cxl latency how much is added at hc34. <https://www.servethehome.com/compute-express-link-cxllatency-how-much-is-added-at-hc34/>, 2023.

- [13] Nvidia gh200 grace hopper superchip architecture. <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper>, 2023.
- [14] 2025. Personal communication with Mingxing Zhang regarding the experimental setup in [72].
- [15] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. Can far memory improve job throughput? In *EuroSys*, 2020.
- [16] Emmanuel Amaro, Zhihong Luo, Amy Ousterhout, Arvind Krishnamurthy, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. Remote memory calls. In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks*, HotNets '20, page 3844, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450381451. doi: 10.1145/3422604.3425923. URL <https://doi.org/10.1145/3422604.3425923>.
- [17] Cristiana Amza, Alan L Cox, Sandhya Dwarkadas, Pete Keleher, Honghui Lu, Ramakrishnan Rajamony, Weimin Yu, and Willy Zwaenepoel. Treadmarks: Shared memory computing on networks of workstations. *Computer*, 29(2):18–28, 1996.
- [18] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2023.
- [19] John K Bennett, John B Carter, and Willy Zwaenepoel. Munin: Distributed shared memory based on type-specific memory coherence. In *Proceedings of the second ACM SIGPLAN symposium on Principles & practice of parallel programming*, pages 168–176, 1990.
- [20] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosz, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. The CacheLib caching engine: Design and experiences at scale. In

- 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 753–768. USENIX Association, November 2020. ISBN 978-1-939133-19-9.
- [21] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry Li, et al. {TAO}::{Facebooks} distributed data store for the social graph. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 49–60, 2013.
 - [22] Qingchao Cai, Wentian Guo, Hao Zhang, Divyakant Agrawal, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, Yong Meng Teo, and Sheng Wang. Efficient distributed memory management with rdma and caching. *Proceedings of the VLDB Endowment*, 11(11):1604–1617, 2018.
 - [23] Irina Calciu, Ivan Puddu, Aasheesh Kolli, Andreas Nowatzky, Jayneel Gandhi, Onur Mutlu, and Pratap Subrahmanyam. Project pberry: Fpga acceleration for remote memory. HotOS ’19, pages 127–135, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367271.
 - [24] Irina Calciu, M. Talha Imran, Ivan Puddu, Sanidhya Kashyap, Hasan Al Maruf, Onur Mutlu, and Aasheesh Kolli. Rethinking software runtimes for disaggregated memory. In *ASPLOS*, pages 79–92, 2021.
 - [25] Roy Campbell, Garry Johnston, and Vincent Russo. Choices (class hierarchical open interface for custom embedded systems). *ACM SIGOPS Operating Systems Review*, 21(3):9–17, 1987.
 - [26] John B Carter, John K Bennett, and Willy Zwaenepoel. Implementation and performance of munin. *ACM SIGOPS Operating Systems Review*, 25(5):152–164, 1991.
 - [27] John B Carter, John K Bennett, and Willy Zwaenepoel. Techniques for reducing consistency-related communication in distributed shared-memory systems. *ACM Transactions on Computer Systems (TOCS)*, 13(3):205–243, 1995.

- [28] CBRE. North america data center trends h2 2021. <https://www.cbre.com/insights/reports/north-america-data-center-trends-h2-2021>, 2022.
- [29] Satish Chandra, Vijay Saraswat, Vivek Sarkar, and Rastislav Bodik. Type inference for locality analysis of distributed data structures. In *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming*, pages 11–22, 2008.
- [30] Lei Chen, Shi Liu, Chenxi Wang, Haoran Ma, Yifan Qiao, Zhe Wang, Chenggang Wu, Youyou Lu, Xiaobing Feng, Huimin Cui, Shan Lu, and Harry Xu. A tale of two paths: Toward a hybrid data plane for efficient Far-Memory applications. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 77–95, July 2024.
- [31] Christopher Clark, Keir Fraser, Steven Hand, Jacob Gorm Hansen, Eric Jul, Christian Limpach, Ian Pratt, and Andrew Warfield. Live migration of virtual machines. In *2nd Symposium on Networked Systems Design & Implementation (NSDI 05)*, Boston, MA, May 2005. USENIX Association. URL <https://www.usenix.org/conference/nsdi-05/live-migration-virtual-machines>.
- [32] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, page 143154, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781450300360. doi: 10.1145/1807128.1807152. URL <https://doi.org/10.1145/1807128.1807152>.
- [33] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. An introduction to the compute express link (cxl) interconnect. *ACM Comput. Surv.*, 56(11), July 2024.
- [34] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. In *6th Symposium on Operating Systems De-*

- sign & Implementation (OSDI 04)*, San Francisco, CA, December 2004. USENIX Association. URL <https://www.usenix.org/conference/osdi-04/mapreduce-simplified-data-processing-large-clusters>.
- [35] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. Low-latency graph streaming using compressed purely-functional trees. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2019. Association for Computing Machinery, 2019. ISBN 9781450367127.
 - [36] Fred Douglass and John Ousterhout. Transparent process migration: Design alternatives and the sprite implementation. *Software: Practice and Experience*, 21(8):757–785, 1991.
 - [37] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. FaRM: Fast remote memory. In *NSDI*, pages 401–414, 2014.
 - [38] Tarek El-Ghazawi, William Carlson, Thomas Sterling, and Katherine Yelick. *UPC: distributed shared memory programming*. John Wiley & Sons, 2005.
 - [39] Brett D Fleisch. Distributed shared memory in a loosely coupled distributed system. *ACM SIGCOMM Computer Communication Review*, 17(5):317–327, 1987.
 - [40] Christine H. Flood, Roman Kennke, Andrew Dinn, Andrew Haley, and Roland Westrelin. Shenandoah: An open-source concurrent compacting garbage collector for openjdk. In *PPPJ*, pages 13:1–13:9, 2016.
 - [41] Peter X. Gao, Akshay Narayan, Sagar Karandikar, Joao Carreira, Sangjin Han, Rachit Agarwal, Sylvia Ratnasamy, and Scott Shenker. Network requirements for resource disaggregation. In *OSDI*, pages 249–264, 2016.
 - [42] GenZ. Genz consortium. <http://genzconsortium.org/>, 2019.

- [43] Google. Google’s fast compressor/decompressor. <https://github.com/google/snappy>, 2020.
- [44] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. Direct access, High-Performance memory disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 287–294, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-29-65. URL <https://www.usenix.org/conference/atc22/presentation/gouk>.
- [45] Grand View Research Inc. Servers market share. <https://www.grandviewresearch.com/industry-analysis/server-market>, 2022.
- [46] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G. Shin. Efficient memory disaggregation with infiniswap. In *NSDI*, pages 649–667, 2017.
- [47] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, and Reetuparna Das. PIM is all you need: A CXL-enabled GPU-free system for large language model inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS ’25, page 862881, 2025.
- [48] Zhiyuan Guo, Yizhou Shan, Xuhao Luo, Yutong Huang, and Yiying Zhang. Clio: A hardware-software co-designed disaggregated memory system. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’22, page 417433, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392051. doi: 10.1145/3503222.3507762. URL <https://doi.org/10.1145/3503222.3507762>.
- [49] Zhiyuan Guo, Zijian He, and Yiying Zhang. Mira: A program-behavior-guided far memory system. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP ’23, page 692708, 2023.

- [50] David B Gustavson. The scalable coherent interface and related standards projects. *IEEE micro*, 12(1):10–22, 1992.
- [51] h2oai. Database-like ops benchmark. <https://github.com/h2oai/db-benchmark>, 2021.
- [52] Sangjin Han, Norbert Egi, Aurojit Panda, Sylvia Ratnasamy, Guangyu Shi, and Scott Shenker. Network support for resource disaggregation in next-generation datacenters. In *HotNets*, pages 10:1–10:7, 2013.
- [53] Riyaz Haque and Jens Palsberg. Type inference for place-oblivious objects. In *29th European Conference on Object-Oriented Programming (ECOOP 2015)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2015.
- [54] F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.
- [55] Bingsheng He, Wenbin Fang, Qiong Luo, Naga K. Govindaraju, and Tuyong Wang. Mars: A mapreduce framework on graphics processors. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques, PACT '08*, page 260269, New York, NY, USA, 2008. Association for Computing Machinery. ISBN 9781605582825.
- [56] Hewlett-Packard. The machine: A new kind of computer. <https://www.hpl.hp.com/research/systems-research/themachine/>.
- [57] IDC Corporate. Servers market share. <https://www.idc.com/promo/servers>, 2022.
- [58] Intel Corporation. Transactional Synchronization Extensions. In *Intel® 64 and IA-32 Architectures Software Developers Manual Volume 1: Basic Architecture*, pages 16–1, Santa Clara, CA, 2021. Intel Corporation. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.

- [59] Intel Corporation. XBEGIN. In *Intel® 64 and IA-32 Architectures Software Developer’s Manual Volumes 2A, 2B, 2C, and 2D: Instruction Set Reference, A-Z*, pages 5–611, Santa Clara, CA, 2021. Intel Corporation. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [60] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. CXL-ANNS: Software-Hardware collaborative memory disaggregation and computation for Billion-Scale approximate nearest neighbor search. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 585–600, 2023.
- [61] Xu Ji, Chao Wang, Nosayba El-Sayed, Xiaosong Ma, Youngjae Kim, Sudharshan S Vazhkudai, Wei Xue, and Daniel Sanchez. Understanding object-level memory access patterns across the spectrum. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2017.
- [62] Stefanos Kaxiras, David Klaftenegger, Magnus Norgren, Alberto Ros, and Konstantinos Sagonas. Turning centralized coherence and distributed critical-section execution on their head: A new approach for scalable distributed shared memory. In *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing*, pages 3–14, 2015.
- [63] Kimberly Keeton. The Machine: An architecture for memory-centric computing. In *ROSS*, 2015.
- [64] Ana Klimovic, Heiner Litz, and Christos Kozyrakis. ReFlex: Remote flash $\approx$ local flash. In *ASPLOS*, pages 345–359, 2017.
- [65] Pradeep Kumar and H. Howie Huang. GraphOne: A data store for real-time analytics on evolving graphs. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 249–263, Boston, MA, February 2019. USENIX Association. ISBN 978-1-939133-09-0.

- [66] Jérôme Kunegis. Konect: the koblenz network collection. In *Proceedings of the 22nd international conference on world wide web*, pages 1343–1350, 2013.
- [67] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. What is Twitter, a social network or a news media? In *WWW '10: Proceedings of the 19th international conference on World wide web*, pages 591–600, New York, NY, USA, 2010. ACM. ISBN 978-1-60558-799-8. doi: <https://doi.acm.org/10.1145/1772690.1772751>.
- [68] Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, Greg Thelen, Kamil Adam Yurtsever, Yu Zhao, and Parthasarathy Ranganathan. Software-defined far memory in warehouse-scale computers. In *ASPLOS*, pages 317–330, 2019.
- [69] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*, CGO '04, page 75, USA, 2004. IEEE Computer Society. ISBN 0769521029.
- [70] KyungSoo Lee, Sohyun Kim, Joohee Lee, Donguk Moon, Rakie Kim, Honggyu Kim, Hyeongtak Ji, Yunjeong Mun, and Youngpyo Joo. Improving key-value cache performance with heterogeneous memory tiering: A case study of cxl-based memory expansion. *IEEE Micro*, pages 1–11, 2024.
- [71] Sergey Legtchenko, Hugh Williams, Kaveh Razavi, Austin Donnelly, Richard Black, Andrew Douglas, Nathanaël Cheriére, Daniel Fryer, Kai Mast, Angela Demke Brown, Ana Klimovic, Andy Slowey, and Antony Rowstron. Understanding rack-scale disaggregated storage. In *HotStorage*, 2017.
- [72] Huaicheng Li, Daniel S. Berger, Stanko Novakovic, Lisa Hsu, Dan Ernst, Pantea Zardoshti, Monish Shah, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo

- Bianchini. First-generation memory disaggregation for cloud platforms, 2022. URL <https://arxiv.org/abs/2203.00241>.
- [73] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: Cxl-based memory pooling systems for cloud platforms. In *ASPLOS*, pages 574–587, 2023.
- [74] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: Cxl-based memory pooling systems for cloud platforms. *ASPLOS 2023*, pages 574–587, 2023.
- [75] Kai Li. Ivy: A shared virtual memory system for parallel computing. *ICPP (2)*, 88: 94, 1988.
- [76] Kai Li and Paul Hudak. Memory coherence in shared virtual memory systems. *ACM Trans. Comput. Syst.*, 7(4):321–359, November 1989. ISSN 0734-2071.
- [77] Kai Li and Paul Hudak. Memory coherence in shared virtual memory systems. *ACM Transactions on Computer Systems (TOCS)*, 7(4):321–359, 1989.
- [78] Yong Li, Rami Melhem, Ahmed Abousamra, and Alex K. Jones. Compiler-assisted data distribution for chip multiprocessors. In *2010 19th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 501–512, 2010.
- [79] Chao Liu, Hung-chih Yang, Jinliang Fan, Li-Wei He, and Yi-Min Wang. Distributed nonnegative matrix factorization for web-scale dyadic data analysis on mapreduce. In *Proceedings of the 19th international conference on World wide web*, pages 681–690, 2010.
- [80] Haoran Ma, Shi Liu, Chenxi Wang, Yifan Qiao, Michael D. Bond, Stephen M. Blackburn, Miryung Kim, and Guoqing Harry Xu. Mako: A low-pause, high-throughput

- evacuating collector for memory-disaggregated datacenters. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, page 92107, 2022.
- [81] Haoran Ma, Shi Liu, Chenxi Wang, Yifan Qiao, Michael D. Bond, Stephen M. Blackburn, Miryung Kim, and Guoqing Harry Xu. Mako: A low-pause, high-throughput evacuating collector for memory-disaggregated datacenters. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2022, page 92107, 2022.
- [82] Haoran Ma, Yifan Qiao, Shi Liu, Shan Yu, Yuanjiang Ni, Qingda Lu, Jiesheng Wu, Yiyang Zhang, Miryung Kim, and Harry Xu. Drust: Language-guided distributed shared memory with fine granularity, full transparency, and ultra efficiency. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [83] Yandong Mao, Robert Morris, and M. Frans Kaashoek. Optimizing MapReduce for Multicore Architectures. Technical report, Massachusetts Institute of Technology, 5 2010.
- [84] Hasan Al Maruf and Mosharaf Chowdhury. Effectively prefetching remote memory with Leap. In *USENIX ATC*, pages 843–857, 2020.
- [85] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 742–755, 2023.
- [86] Ronald G Minnich and David J Farber. The mether system: Distributed shared memory for sunos 4.0. *Technical Reports (CIS)*, page 332, 1993.

- [87] Hossein Moein. C++ dataframe for statistical, financial, and ml analysis. <https://github.com/hosseinmoein/DataFrame>, 2020.
- [88] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, 2018.
- [89] Jacob Nelson, Brandon Holt, Brandon Myers, Preston Briggs, Luis Ceze, Simon Kahan, and Mark Oskin. {Latency-Tolerant} software distributed shared memory. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 291–305, 2015.
- [90] Jacob Nelson, Brandon Holt, Brandon Myers, Preston Briggs, Luis Ceze, Simon Kahan, and Mark Oskin. Latency-tolerant software distributed shared memory. In *USENIX ATC*, pages 291–305, 2015.
- [91] J Nieplocha, R Harrison, M Krishnan, B Palmer, and V Tipparaju. Combining shared and distributed memory models: Evolution and recent advancements of the global array toolkit. In *proceedings of POHLL’2002 workshop of ICS-2002, NYC*, 2002.
- [92] Jaroslaw Nieplocha, Robert J Harrison, and Richard J Littlefield. Global arrays: A portable" shared-memory" programming model for distributed memory computers. In *Supercomputing’94: Proceedings of the 1994 ACM/IEEE conference on Supercomputing*, pages 340–349. IEEE, 1994.
- [93] Jaroslaw Nieplocha, Robert J Harrison, and Richard J Littlefield. Global arrays: A nonuniform memory access programming model for high-performance computers. *The Journal of Supercomputing*, 10:169–189, 1996.
- [94] Inc NumFOCUS. Pandas. <https://pandas.pydata.org/>, 2022.

- [95] NVIDIA. Nvidia connectx infiniband adapters. <https://www.nvidia.com/en-sg/networking/infiniband-adapters>, 2023.
- [96] Nvidia. Virtual gpu (vgpu) | nvidia. <https://www.nvidia.com/en-us/data-center/virtual-solutions/>.
- [97] Oracle. The java virtual machine. <https://www.java.com/en/download>, 2023.
- [98] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *NSDI*, pages 361–378, 2019.
- [99] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical report, Stanford infolab, 1999.
- [100] Guilherme Piccoli, Henrique N. Santos, Raphael E. Rodrigues, Christiane Pousa, Edson Borin, and Fernando M. Quintão Pereira. Compiler support for selective page migration in numa architectures. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation, PACT '14*, page 369380, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450328098.
- [101] Yifan Qiao, Chenxi Wang, Zhenyuan Ruan, Adam Belay, Qingda Lu, Yiyang Zhang, Miryung Kim, and Guoqing Harry Xu. Hermit: Low-Latency, High-Throughput, and transparent remote memory via Feedback-Directed asynchrony. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 181–198, Boston, MA, April 2023. USENIX Association. ISBN 978-1-939133-33-5.
- [102] Colby Ranger, Ramanan Raghuraman, Arun Penmetsa, Gary Bradski, and Christos Kozyrakis. Evaluating mapreduce for multi-core and multiprocessor systems. In *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, 2007.

- [103] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K. Aguilera, and Adam Belay. AIFM: High-performance, application-integrated far memory. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 315–332, 2020. ISBN 978-1-939133-19-9.
- [104] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K Aguilera, and Adam Belay. {AIFM}:{High-Performance},{Application-Integrated} far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 315–332, 2020.
- [105] Zhenyuan Ruan, Seo Jin Park, Marcos K Aguilera, Adam Belay, and Malte Schwarzkopf. Nu: Achieving {Microsecond-Scale} resource fungibility with logical processes. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1409–1427, 2023.
- [106] Stephen M. Rumble, Ankita Kejriwal, and John Ousterhout. Log-structured memory for dram-based storage. In *Proceedings of the 12th USENIX Conference on File and Storage Technologies*, FAST’14, page 116, USA, 2014. USENIX Association. ISBN 9781931971089.
- [107] Conrad Sanderson. Armadillo: An open source c++ linear algebra library for fast prototyping and computationally intensive experiments. 2010.
- [108] Yizhou Shan, Shin-Yeh Tsai, and Yiying Zhang. Distributed shared persistent memory. In *Proceedings of the 2017 Symposium on Cloud Computing*, pages 323–337, 2017.
- [109] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In *OSDI*, pages 69–87, 2018.
- [110] Yizhou Shan, Will Lin, Ryan Kosta, Arvind Krishnamurthy, and Yiying Zhang. Optimizing hardware-based network computation dags for multiple tenants with supernic. *arXiv preprint arXiv: Arxiv-2109.07744*, 2021.

- [111] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *EuroSys*, 2020.
- [112] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys '20, 2020. ISBN 9781450368827.
- [113] Robert Stets, Sandhya Dwarkadas, Nikolaos Hardavellas, Galen Hunt, Leonidas Kontothanassis, Srinivasan Parthasarathy, and Michael Scott. Cashmere-2l: Software coherent shared memory on a clustered remote-write network. In *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, pages 170–183, 1997.
- [114] Xiaoyuan Su and Taghi M Khoshgoftaar. A survey of collaborative filtering techniques. *Advances in artificial intelligence*, 2009(1):421425, 2009.
- [115] Konstantin Taranov, Salvatore Di Girolamo, and Torsten Hoefler. Corm: Compactable remote memory over rdma. In *Proceedings of the 2021 International Conference on Management of Data*, pages 1811–1824, 2021.
- [116] Abhishek Verma, Luis Pedrosa, Madhukar R. Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, Bordeaux, France, 2015.
- [117] Lluís Vilanova, Lina Maudlej, Shai Bergman, Till Miemietz, Matthias Hille, Nils Asmussen, Michael Roitzsch, Hermann Härtig, and Mark Silberstein. Slashing the disaggregation tax in heterogeneous data centers with fractos. In *Proceedings of the Seventeenth European Conference on Computer Systems*, EuroSys '22, page 352367, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450391627. doi: 10.1145/3492321.3519569. URL <https://doi.org/10.1145/3492321.3519569>.

- [118] Keval Vora, Rajiv Gupta, and Guoqing Xu. Kickstarter: Fast and accurate computations on streaming graphs via trimmed approximations. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '17*, page 237251, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450344654.
- [119] Chenxi Wang, Huimin Cui, Ting Cao, John Zigman, Haris Volos, Onur Mutlu, Fang Lv, Xiaobing Feng, and Guoqing Harry Xu. Panthera: Holistic memory management for big data processing over hybrid memories. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019*, page 347362, 2019.
- [120] Chenxi Wang, Haoran Ma, Shi Liu, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D. Bond, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. Semeru: A memory-disaggregated managed runtime. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 261–280. USENIX Association, November 2020. ISBN 978-1-939133-19-9. URL <https://www.usenix.org/conference/osdi20/presentation/wang>.
- [121] Chenxi Wang, Haoran Ma, Shi Liu, Yifan Qiao, Jonathan Eyolfson, Christian Navasca, Shan Lu, and Guoqing Harry Xu. MemLiner: Lining up tracing and application for a Far-Memory-Friendly runtime. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 35–53, 2022.
- [122] Chenxi Wang, Yifan Qiao, Haoran Ma, Shi Liu, Yiyang Zhang, Wenguang Chen, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. Canvas: Isolated and adaptive swapping for multi-applications on remote memory. In *NSDI*, 2023.
- [123] Chenxi Wang, Yizhou Shan, Pengfei Zuo, and Huimin Cui. Reinvent cloud software stacks for resource disaggregation. *Journal of Computer Science and Technology*, 38(5):949–969, 2023. ISSN 1860-4749.

- [124] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. Concordia: Distributed shared memory with {In-Network} cache coherence. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 277–292, 2021.
- [125] Zhonghua Wang, Yixing Guo, Kai Lu, Jiguang Wan, Daohui Wang, Ting Yao, and Huatao Wu. Rcmp: Reconstructing rdma-based memory disaggregation via cxl. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024.
- [126] Wei Wei, Dejun Jiang, Sally A. McKee, Jin Xiong, and Mingyu Chen. Exploiting program semantics to place data in hybrid memory. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*, pages 163–173, 2015. doi: 10.1109/PACT.2015.10.
- [127] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. Tmo: Transparent memory offloading in datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '22*, page 609621, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392051. doi: 10.1145/3503222.3507731. URL <https://doi.org/10.1145/3503222.3507731>.
- [128] WMT workshops. Statistical and neural machine translation. <https://statmt.org>, 2011.
- [129] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. Nomad: Non-Exclusive memory tiering via transactional page migration. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 19–35, 2024.
- [130] Jaewon Yang and Jure Leskovec. Friendster social network and ground-truth communities. <https://snap.stanford.edu/data/com-Friendster.html>, 2012.

- [131] Katherine Yelick, Dan Bonachea, Wei-Yu Chen, Phillip Colella, Kaushik Datta, Jason Duell, Susan L Graham, Paul Hargrove, Paul Hilfinger, Parry Husbands, et al. Productivity and performance using partitioned global address space languages. In *Proceedings of the 2007 international workshop on Parallel symbolic computation*, pages 24–32, 2007.
- [132] Hangchen Yu, Arthur M. Peters, Amogh Akshintala, and Christopher J. Rossbach. Automatic virtualization of accelerators. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, HotOS '19, page 5865, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367271. doi: 10.1145/3317550.3321423. URL <https://doi.org/10.1145/3317550.3321423>.
- [133] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. HotCloud, page 10, Berkeley, CA, USA, 2010.
- [134] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. Partial failure resilient memory management system for (CXL-based) distributed shared memory. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 658–674, 2023.
- [135] Wei Zhang, Sundaresan Rajasekaran, Shaohua Duan, Timothy Wood, and Mingfa Zhuy. Minimizing interference and maximizing progress for hadoop virtual machines. *SIGMETRICS Perform. Eval. Rev.*, 42(4):62–71, 2015.
- [136] Yuhong Zhong, Daniel S. Berger, Carl Waldspurger, Ryan Wee, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D. Hill, Mosharaf Chowdhury, and Asaf Cidon. Managing memory tiers with CXL in virtualized environments. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 37–56, Santa Clara, CA, 2024.